

云计算独立任务及关联任务调度算法研究



重庆大学硕士学位论文 (学术学位)

学生姓名：张晓磊

指导教师：王 波 副教授

专 业：计算机应用技术

学科门类：工 学

重庆大学计算机学院

二〇一四年四月

Study on scheduling algorithm of the independent and associated tasks for cloud computing



A Thesis Submitted to Chongqing University
in Partial Fulfillment of the Requirement for the
Master's Degree of Engineering

By
Zhang Xiaolei

Supervised by Ass. Prof. Wang Bo
Specialty: Computer Applications Technology

College of Computer of Chongqing University,
Chongqing, China
April, 2014

摘 要

云数据中心的任务调度是云计算应用的核心问题，是云计算得以大规模应用和提高系统性能的关键技术。先进的任务调度，对于提高云服务提供商计算资源的利用效率、节约能源、提高资源共享、和降低运营成本都具有极大意义，值得深入系统地学习研究。

本文的主要工作：

(1) 研究了云计算的关键技术，详细对比分析了目前学术界热点研究的几种任务调度算法，并针对云计算中独立任务和相关联任务的调度算法进行了深入研究。从中发现了一些缺陷，并提出了相应的改进方法。

(2) 针对云计算中的独立任务调度，综合考虑用户满意度和云服务提供商收益，提出了一种融合粒子群和遗传算法，应用到了云调度中。首先，对云计算系统中的虚拟机进行分类，并引入任务-资源满意度距离、资源综合性能概念；其次，优化粒子群算法初始化粒子操作，提高初始粒子质量；然后，为了克服粒子群算法容易陷入局部最优解的问题，融合遗传算法来扩展粒子群的搜索空间。最后，在云仿真软件Cloudsim上的实验结果表明，相比其他调度算法，该算法能有效提高用户满意度和增加云服务提供商利润，是一种有效的调度算法。

(3) 针对云计算中的相关联任务，在研究了经典调度算法的基础上提出了基于改进优先级和任务复制的调度算法。在云计算的异构环境下，DAG任务图关键路径已经失去表示最迫切需要调度任务的意义。本文算法在确定任务优先级别时综合考虑了任务计算通信代价、任务出度和任务分支值等因素，在选择处理器阶段利用复制冗余任务的手段来提前任务的开始执行时间。通过这两方面的改进，理论证明和实验分析说明，本文算法对比现有调度算法，有效的减少了通信延迟开销，缩短了DAG任务图完成时间。

本论文主要研究了云计算环境下独立任务和相关联任务的调度算法。提出的独立任务算法更适合云数据中心大吞吐量的环境；提出的相关联任务调度算法更适合通信密集型任务调度。该课题的研究为分析和解决云计算任务调度问题提供了新的思路和参考。

关键词：云计算，独立任务，相关联任务，静态调度，动态调度

ABSTRACT

Task scheduling in cloud data center is the core issue of cloud computing, also the key technology for cloud computing's large-scale application and system performance improve. Advanced task scheduling have a great significance for cloud service providers to improve the efficiency of computing resources, save energy, improve resource sharing, and reduce operating costs deserves further systematic study.

The main work of this paper:

(1) Studied the key technologies of cloud computing, analyses several current hot research task scheduling algorithm in academia detailed and comparatively; and researched the independent and associated task scheduling algorithm in cloud computing deeply. we found some deficiencies, and proposed the corresponding method.

(2) For the independent task scheduling, we consider user's satisfaction and cloud service provider's avails synthetically, and then propose a merge scheduling algorithm which combine genetic algorithm with particle swarm optimization. Firstly, we classify the virtual machines in cloud computing system, and introduce the degree of task-resource satisfaction, resource comprehensive performance. Secondly, we optimize particle initialization operations to improve the quality of the initial particle in particle swarm optimization. Thirdly, in order to overcome the defect of particle swarm optimization's easy falling into local optimal solution, we integrate genetic algorithm to extend the search space. Finally, compared with other scheduling algorithm, experimental on cloud simulation software Cloudsim show that this paper algorithm is an effective scheduling algorithm which can effectively improve user's satisfaction and increase cloud service provider's gain.

(3) For the associated task scheduling, under the fundamental of studying classical scheduling algorithm we propose a new one based on task priority and task replication. In heterogeneous environments of cloud computing, critical path in DAG task scheduling has lost its meaning to the most urgent task. In this paper, when determining the priority task, we consider factors such as the task communication cost, the task computing cost, the task out degree and the branching values of task comprehensively. In the processor selection stage, we use the means of replicate redundant tasks to

advance the time of task's beginning execution. Through these two aspects improving, both theoretical and experimental analysis show that this proposed algorithm comparison of existing scheduling algorithms, reducing the cost of communication delay effectively, shorten the complete time of the DAG task.

In this article, we study the independent tasks and the associated tasks scheduling algorithm under the cloud computing environment. The proposed scheduling algorithm of the independent task is more suitable for the cloud data center's large throughput environments; the proposed scheduling algorithm of the associated task is more suitable for communication-intensive task scheduling. Research on the subject provides a reference and new ideas to solve cloud computing task scheduling problem.

Keywords: cloud computing, the independent task, the independent task, static scheduling, dynamic scheduling

目 录

中文摘要.....	I
英文摘要.....	II
1 绪 论.....	1
1.1 课题研究背景和意义.....	1
1.2 国内外研究现状分析.....	3
1.2.1 独立任务调度研究现状.....	3
1.2.2 相关联任务调度研究现状.....	4
1.2.3 主要厂商方案.....	5
1.3 论文的研究内容.....	6
1.4 论文组织结构安排.....	6
2 云计算任务调度技术.....	7
2.1 云计算概述.....	7
2.1.1 云计算中的不同角色.....	7
2.1.2 虚拟化技术.....	7
2.1.3 并行计算架构.....	7
2.1.4 多租赁和按需计费.....	8
2.1.5 云计算与网格计算.....	8
2.2 任务调度概述.....	9
2.2.1 任务调度关键问题.....	9
2.2.2 任务调度流程.....	9
2.3 现有任务调度算法.....	10
2.3.1 独立任务调度算法.....	10
2.3.2 相关联任务调度算法.....	12
2.4 本章小结.....	15
3 基于粒子群遗传算法的独立任务调度算法.....	16
3.1 独立任务调度模型.....	16
3.1.1 独立任务模型.....	16
3.1.2 云计算虚拟机模型.....	17
3.1.3 云计算虚拟机综合性能.....	17
3.1.4 任务-资源满意度距离.....	18
3.2 任务调度需求分析和设计目标.....	18

3.2.1 需求分析	18
3.2.2 设计目标	18
3.3 任务调度具体方案	19
3.3.1 对任务和虚拟机参数归一化处理	19
3.3.2 计算任务最佳匹配虚拟机	20
3.3.3 PSO 与 GA 融合的混合 PSOGA 算法	20
3.4 算法性能分析	22
3.5 本章小结	23
4 基于改进表调度的相关联任务调度算法	24
4.1 DAG 任务建模	24
4.2 异构的云计算环境	26
4.3 基于表调度的改进的任务调度算法	27
4.3.1 算法假设条件	27
4.3.2 问题分析	28
4.3.3 改进的表调度算法	29
4.4 算法描述	32
4.5 实例分析	34
4.6 本章小结	37
5 仿真实验与结果分析	38
5.1 CloudSim 仿真工具简介	38
5.1.1 CloudSim 体系结构	38
5.1.2 CloudSim 工作方式	39
5.2 CloudSim 配置及仿真流程	40
5.2.1 环境配置	40
5.2.2 仿真流程	41
5.3 独立任务调度算法仿真实验与结果分析	42
5.3.1 评价指标	42
5.3.2 实验设计	42
5.3.3 实验结果分析	44
5.4 相关联任务调度算法仿真实验及结果分析	44
5.4.1 实验设置	44
5.4.2 评价指标	45
5.4.3 实验结果分析	45
5.5 本章小结	48

6 总结与展望	49
6.1 工作总结	49
6.2 展望	50
致 谢	51
参考文献	52
附 录	55

1 绪 论

1.1 课题研究背景和意义

自从 2006 年 8 月，谷歌在搜索引擎大会首次提出“云计算”的概念以来，云计算^{[1][2][3]}便迅速进入人们的视野，之后开始受到企业界以及学术界的高度关注。云计算带来了一种新的计算模式和服务模式，其本质是计算和存储资源不再由本地资源提供，而是采用第三方服务商运营的数据中心来提供。

随着网络性能的提高以及智能终端设备的普及，云计算正以越来越快的速度走进人们的生活，人们开始慢慢感受到云计算的魅力。云计算通过网络可以提供的计算与信息服务与应用有很多种，包括计算、存储、网络、服务、软件等多种形式。但云计算也不是一蹴而就的，而是从并行计算技术、网格计算技术^[4]、软件作为服务与虚拟化技术等演进而来，即一脉相承，又有所不同，如图 1-1 所示。

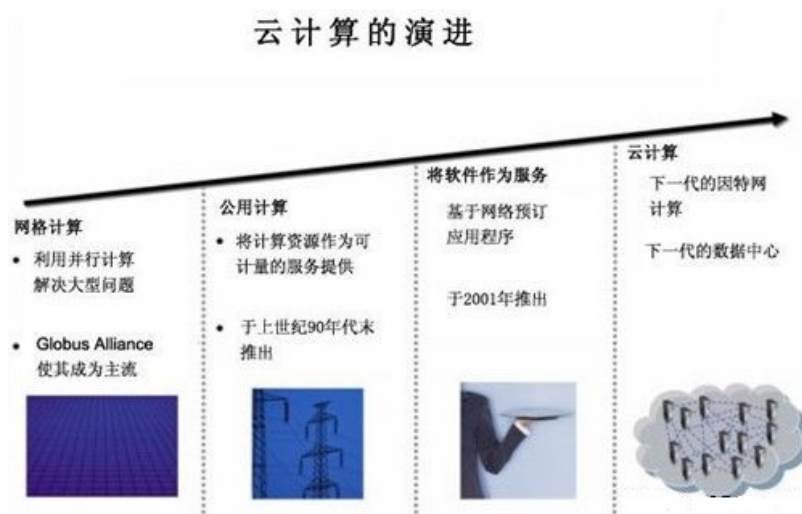


图 1.1 云计算演进过程

Fig. 1.1 The evolution of cloud computing

互联网 web2.0 的高速发展，以及社会信息化进程的不断推进，引发了对海量数据处理的迫切需求，云计算正是在这种背景下应运而生。云计算囊括了计算机界先前的虚拟化技术、大规模数据中心、软件作为服务等先进技术。基于互联网的信息爆炸是驱动云计算产生的主要原因：据统计近几年，数据的生成速度呈简直不可思议，在 2010 年，全世界 50% 以上的公司，每天产生 1TB 以上的数据，其中 5% 的公司，每天产生的数据量竟然超过 50TB。众所周知，数据是现代企业的核心竞争力之一。各个公司都在努力挖掘数据中蕴涵的商机，但是存储和处理数

据需要大量的计算资源。对于如何合理整合这些资源，计算机工程师逐步演化出了“云计算”的思想。

云计算提供对资源的集中统一管理，多个用户之间可以共享资源，提高了资源利用率，这种新的模式人们较容易接受，也更实用。而传统的 IT 架构下，各个企业和组织需要自己购买和维护硬件软件，企业可能由于资金预算问题使得自有资源不能满足企业存储和计算的需求，也可能为了满足峰值计算需求，使得企业在正常情况下浪费了大量资源。据统计数据显示，大多数企业的数据中心和集群平均资源利用率不到 30%。而与此同时，企业用于管理维护自身的 IT 基础设施的开销则在逐年增长，能耗开销也在不断加大。

云计算相比以前的计算模式，具有如下新的特征：

（1）虚拟化，云计算平台是以虚拟化技术为基础，是支撑云计算的重要技术基石。对资源利用率的提高，系统安全性和可靠性的提高，都有很大好处。

（2）资源动态伸缩性，解决了资源利用率和系统可用性之间的矛盾。当系统负载较高时，创建更多的虚拟服务器来共同提供服务，减轻了原有服务器的负载；当系统负载较低时，通过减少虚拟服务器数量来提升资源利用率。

（3）按需服务，云服务和云计算平台可以根据客户的实际需求来提供。此模式避免了中小企业创业初期大量硬件资源投入风险，只是根据客户实际需要，使用一定量的计算存储资源。当客户不再需要使用时就释放掉这些资源，就不用再为这部分资源付费。

（4）规模化效应，云计算都是建立在大规模的数据中心基础上，例如 Amazon 的数据中心，这样就可以利用规模化效应，降低客户的租用费用，从而吸引更多的客户使用云计算。

（5）高可靠性，云数据中心都是采用多数据冗余备份来保证用户数据的可靠性，并且实时监控资源节点的运行状态，及动态迁移故障节点的服务，保证系统整体性能。

（6）动态可定制，客户可以根据自己的需求，在使用时动态定制和部署虚拟设备，云基础设施服务商给予特权来访问虚拟服务器。

伴随云计算优势的是也带来了一系列的挑战：

（1）安全性问题，对于数据安全要求高的企业（如银行、军事等），客户信息安全级别要求极高。云计算能否保证数据安全性是一个业界普遍关心的问题。

（2）可靠性问题，在规模庞大的云数据中心中，云计算服务提供商要同时保证用户数据和应用平台的可靠性，如何保证极高的可靠性需要很好的解决方案。

（3）管理性问题，云计算平台的管理非常复杂和庞大，如何高效的监控系统资源，动态调度和部署任务以及管理客户等都面临着极大的挑战。

对于云服务提供商，他们所关心的是如何提升对用户的服务质量、提高数据中心吞吐率以及减少运营成本。这就涉及到了任务调度问题，任务调度被认为是影响数据中心的新技术。云服务提供商的数据中心情况各不相同，因此需要针对各自实际情况，设定自己的任务调度策略。目前，云计算调度技术研究还处于发展阶段，还没有统一的标准和规范，现有的调度策略也不能直接拿来用于云计算任务调度。因此，研究适应云计算环境的任务调度策略，具有较大的实用意义。

1.2 国内外研究现状分析

任务调度问题是云数据中心能否高效运行的关键问题之一。云计算是网格计算的进一步发展，可以借鉴网格计算中现有调度算法，为云计算环境下的任务调度提供设计参考，但由于云计算具有新的特征，并不能直接拿来使用。

云计算调度策略首先要确定调度目标，例如使得任务集合的完成时间最短且资源得到充分利用。一个好的调度算法，要求能在较短时间内得到优化结果，并且自身又不能消耗太多的资源。分布式环境下任务调度是一个经典的调度问题，调度问题被证明是 NP 难问题^[5]，大量的学者投入了许多精力来解决这一问题^[6]。

提交到云数据中心的任务，需要根据云编程模型，分解成子任务后才进行调度。在云计算的分布式集群环境中，一般把研究的任务调度分为独立任务调度和相关联任务调度两大类。根据任务间依赖关系的不同，下面分两个部分来介绍国内外任务调度的研究现状。

1.2.1 独立任务调度研究现状

对于云计算中的独立任务，着重研究如何将独立任务集合在一个分布式的计算集群上调度，一般它追求的目标是整个系统性能最佳，或者达到特定的调度目标。因此，任务调度策略需要充分考虑制约任务的各种因素，才能为用户任务分配到最合适的资源，以达到设定的调度目标。从调度算法的特点可以分为传统算法和启发式智能调度算法：

(1) 传统调度算法：有 FIFO 算法、最早截止时间优先算法、轮转调度算法、比例公平调度、Min-Min 调度算法、Min-Max 算法等，这些算法的优点是实现简单，复杂度较低且实用性强，缺点是性能不佳^[7]。不少学者基于这些算法进行了改进，例如文献[8]结合了 Min-Min 和 Max-Min 算法，并利用任务期望完成时间的标准偏差进行改进，取得了较好的效果。

(2) 启发式智能调度算法：把一批独立任务组成的任务集调度到云计算的集群环境中，相当于一个优化问题，因此，可以采用智能调度算法来尝试解决。目前已经有文献中出现，将遗传算法^[9]、博弈论算法^[10]及粒子群算法^[11]等调度方法进行改进，用于云计算环境下的任务调度，并取得较好的性能，但是缺点是求解

耗时较长，过程比较复杂，实用性不强。

从调度算法的调度目标来讲，又可以分为以性能为中心的调度、以服务质量为中心的调度、以提高资源利用率为中心的调度、以经济为中心的调度等。

(1) 以性能为中心的调度：该类调度算法就是以任务跨度最优或者任务完成时间最早作为调度目标。李易峰等人^[12]提出了一个具有双适应度的 DFGA 遗传算法，调度目标是任务的平均完成时间最小；汤小春等^[13]提出了基于元区间的最佳分配决策算法，通过估计待选资源参数，选择满足要求的元区间；华夏渝^[14]首先估计可用资源节点的质量，然后通过分析带宽、响应时间等因素对分配的影响，再利用蚁群算法得到最优的资源分配解。

(2) 以服务质量为中心的调度：该类调度算法增加了对用户服务质量的考虑，而不仅仅只以系统性能为调度目标。何晓珊等^[15]基于 QoS 修改了 Min-min 调度算法，根据用户 QoS 需求对系统吞吐率进行优化，获得了一定的性能改进。孙大为等^[16]根据用户应用偏好对 QoS 中的用户效用进行量化，并结合免疫克隆算法，给出了多维 QoS 优化的适应度函数，改进了算法的收敛能力，提高了算法的性能。

(3) 以提高资源利用率为中心的调度：为虚拟机动态分配物理资源，达到减少使用物理资源和提高资源利用率。Fabien Hermenier 等人^[17]提出了适用同构集群的 Entropy 资源管理器，利用约束规划提升性能，并考虑了虚拟机迁移开销。Entropy 资源管理器分配子任务到节点时，利用约束规划往往能获得全局最优解，要优于基于本地最优的启发式方法。Jean-Marc Menaud 和 Hien Nguyen 等人^{[18][19]}对云环境中虚拟机资源的管理提出了一些动态调度方法，并把这些问题的转化为约束满足问题，综合考虑了 SLA 级别和执行费用来优化全局效用函数。

(4) 以经济原则为中心的调度：云计算的商业计算特性，收益成为了云服务提供商不得不关注的重要因素。Rajkumar Buyya^[20]等人提出基于经济模型资源调度方法，通过 SLA 资源分配器来实现资源提供者与资源使用者之间的协商，实现资源的优化分配。Deelman 等人^[21]利用亚马逊收费标准和一个真实的天文学应用，仿真实验了不同执行方法和资源提供方案下的性价比。Assuncao 等人^[22]研究了企业组织利用云服务提供商基础设施扩展企业本地基础设施的收益问题。

1.2.2 相关联任务调度研究现状

通常使用 DAG 任务图来描述相关联任务。相关联任务特点是任务的所有前驱节点执行完成后，该任务才能开始执行。对相关联任务调度算法，可分为静态调度算法与动态调度算法。若根据 DAG 任务图，可以得知各任务在每个处理机上的计算代价及任务间的传输代价，同时采用非抢占式的调度，这类算法称为静态调度算法。静态调度算法具有简单实用、执行开销低等特点。动态调度算法是根据系统资源节点的实时运行状态，在 DAG 任务图执行期间才决定选择哪个就绪任务

来执行。动态调度算法由于具有很多不确定因素，开销一般较大。传统的静态调度策略主要分以下四类：

(1) 表调度^[23]，首先计算 DAG 任务图中的任务优先级，根据优先级构造一个调度队列，然后将调度队列分配到处理机集合上。在实用性上，该类算法超过了其他种类的启发式调度算法，具有较低的复杂性和较好的调度性能。目前主要有 HEFT、MCP、ETF、DLS^[24]、HLFET、SHN 等。

(2) 聚簇调度，首先把 DAG 任务图划分为簇，然后合并划分好的簇，直到簇的数目和目前可用处理机数目一致。目前主要的聚簇算法有 LC、EZ、MD、DSC、DCP^[25]等。

(3) 基于任务复制的调度，主要思路是冗余复制某些任务到一些处理机上，以减少通信开销。基于任务复制的调度算法主要有 DSH^[26]，CPFD，Bottom-UP-Down，TDSD，Duplication First and Reduction Next 等。

(4) 基于随机搜索技术的调度，主要思想是结合从以前的结果中所获得的经验和一些随机特征来产生新的结果。此类方法可以产生较好的调度结果，但是复杂度较高。主要利用遗传算法、模拟退火算法、局部搜索技术等智能寻优算法。

1.2.3 主要厂商方案

(1) Amazon 调度策略

Amazon 调度策略结合了成本优先、负载均衡、高可靠性、满足不同用户需求等方式。通过预先配置典型虚拟机来加快响应速度，将客户分为立即使用客户和预定客户，成本优先采用不同地区不同成本的收费调度策略，负载均衡采用轮转调度算法。

(2) IBM 调度策略

IBM 的调度目标是性能优先，即尽最大努力满足客户需求。IBM 资源调度管理采用预先配置好虚拟机或提供选择项模板，让用户在线选择，通过 Tivoli Enterprise 门户网站查看每个受监控资源的动态详细信息，用户选择后，系统就会启动一个过程来构建服务器，该过程完全自动。

(3) HP 调度策略

HP 主要考虑成本优先和负载均衡调度策略。HP 有自己完整的成本模型，但尚未看到公开文献介绍 HP 最小化成本策略，负载均衡通过实时监控虚拟机的 CPU、存储和网络资源的利用率实现虚拟机的自动配置和动态迁移。

(4) Hadoop 调度策略

Hadoop 的基本策略包括公平分配、负载均衡、就近选择节点、提高可靠性（备份+动态调整）。Hadoop 采用的主从式任务调度模式，利用 JobTracker 主节点来控制整个云计算系统的任务调度，其余称为 TaskTracker 的节点在空闲时，通过向

JobTracker 主节点申请分配任务执行。主节点通过查询从节点机制动态分配任务，以实现负载均衡，总是优先寻找距离客户数据最近的从节点，以减少不必要的数据网络传输。对数据采用分割备份，在多个节点上备份数据。

1.3 论文的研究内容

本论文的研究内容主要包括：

(1) 本文通过查阅大量文献资料，学习了云计算下的关键技术、任务调度技术等相关内容，并针对独立任务和相关联任务调度策略分别进行了研究，对现有调度算法存在的问题进行了分析。

(2) 对云计算独立任务调度策略进行了研究，综合考虑用户满意度和云服务提供商收益，提出了一种融合粒子群算法和遗传算法的调度策略。首先，云计算系统中的虚拟机进行分类，并引入任务-资源满意度距离、资源综合性能概念；然后，优化粒子群算法初始化粒子操作，提高初始粒子质量；最后，为了克服粒子群算法容易陷入局部最优解的问题，融合遗传算法来扩展粒子群的搜索空间。

(3) 对云计算相关联任务调度策略进行了研究。在研究了经典调度算法的基础上提出了基于改进优先级和任务复制的调度算法。在构造任务队列时提出了新的计算方法，试图构造出更好的任务队列，在为任务分配处理机时，利用处理机空闲时间槽，冗余复制当前任务的满足约束条件的关键父任务，减少任务间的通信代价，使当前任务节点提早开始执行，从而缩短整个 DAG 任务图的完成时间。

1.4 论文组织结构安排

本论文研究的主题是“云计算环境下独立任务和相关联任务的调度问题”，共分六个章节进行阐述，具体安排如下：

第一章是论文绪论部分，首先，介绍了本课题的研究背景和意义，任务调度的国内外研究现状。然后，详细介绍了本论文的研究工作。

第二章对云计算相关技术和任务调度技术进行了概述，然后，详细介绍了一些独立任务和相关联任务的调度算法，并分析了存在的问题。

第三章对云计算独立任务调度策略进行了研究，综合考虑用户满意度和云服务提供商收益，提出了一种融合粒子群算法和遗传算法的调度策略。

第四章对云环境下的相关联任务调度算法进行了研究，改进了经典表调度算法的任务优先级别计算方法，并融合了任务冗余复制技术，通过两方面改进，缩短了任务的完成时间，并通过一个实例进行了分析。

第五章主要是对第三章和第四章提出的改进调度算法进行仿真实验，并进行了性能分析，通过实验验证上述算法的有效性。

第六章是对本文研究内容的总结与展望。

2 云计算任务调度技术

2.1 云计算概述

2.1.1 云计算中的不同角色

(1) 云提供商：处于云计算产业链的核心位置 and 上流，提供云所需的软硬件设备和解决方案，需要具有丰富的软件、硬件和行业经验，为其他角色提供服务。

(2) 云服务提供商：利用云提供商的基础平台提供云计算服务，需要和云提供商密切合作（或自行构建云环境）。

(3) 企业用户：数量庞大的发展中的中小型企业是云计算产业链中的用户。可以按照企业发展的实际需求向云提供商和云服务提供商租用云平台或者构建小型私有云。

(4) 个人用户：个人用户将主要通过 PC 端浏览器、移动智能终端等设备使用云服务提供商提供的服务。用户不需要再购买高性能计算机来运行各种应用软件，也不需要对应应用软件进行维护和升级，从而大大减少用户端系统的成本与安全漏洞。

2.1.2 虚拟化技术

虚拟化技术是将各种计算及存储资源整合和高效利用的关键技术，是云计算的核心组成部分。从目标层次上可分为向上和向下虚拟化，向上虚拟化是整合多个物理资源对外提供统一的虚拟化资源，而向下虚拟化通过分割物理资源对外提供多个互相隔离的虚拟化环境。

云数据中心大量采用了虚拟化技术，使得整个集群系统的不同层面的硬件软件资源一一隔开，消除了数据中心中各种物理设备的异构性以及软件系统的差异。将大规模的数据中心资源以 IT 资源池的形式呈现，最终实现了跨系统的资源调度和资源集中管理。这种资源和服务的统一管理，极大地方便了用户对系统的感知、查询和使用。

2.1.3 并行计算架构

在 1991 年，要达到 10Gflops 的计算能力，需要一台成本约四千万美元的 Cray 超级计算机。而今天，伴随着硬件水平的发展，要达到这样的计算能力，只需合并几台普通的 x64 计算机即可，其成本仅为 4000 美元，硬件已经不再是制约处理能力的第一要素。然而，简单的把几台计算机组合起来，并不等于获得了相应的计算能力。

云计算的数据中心拥有大量的服务器资源，数据中心通过完善的网络计算架构，将服务器资源相互连接，保证服务器资源节点可以协同工作，以结构化的方

式将服务器资源整合在一起，实现并行计算架构，为用户提供高性能的数据处理能力和数据分析能力。对于用户来说，只需要按照约定的编程框架提交待处理作业和相关数据，云计算系统能自动完成分布式计算所必须的其他工作，例如数据分割，中间数据的传输，多机环境下任务执行，作业调度以及聚合输出数据。

2.1.4 按需计费和多租赁

云计算中心在对资源和服务进行统一调配的基础上，将 IT 基础设施、云计算平台、云应用等都以服务的形式通过互联网提供给客户使用。客户只需要为他所使用的那部分计算资源或者服务付费，而无需事先投入大量资金从头到尾地去建设自己的数据中心和 IT 支撑体系，无需自行面对支撑服务的各种复杂 IT 技术问题，更不需要负担日益高昂的数据中心管理成本，从而大量节省设备投资和后期的运维管理费用，满足了企业期待低成本专业化运营的需求。

云数据中心资源池中的资源都是通用的和多用户可共享的。云服务提供商还可以使用 SLA 设置等手段，根据实际业务需求和应用特点，通过自定义策略来对整体系统的性能和安全性进行优化。从而提供面向不同使用需求和不同用户群的差异化和多样化服务。并且，云计算中心应用高可用、数据冗余、负载平衡、备份和容灾等多种手段保证系统的安全可靠运行和用户数据的安全性，以实现用户多租赁。

2.1.5 云计算与网格计算

网格计算在云计算概念之前提出，作为一个热门领域已有十余年的历史。网格计算依托专用网或者互联网，将不同地域的多个空闲计算机资源组织起来，然后，通过系统资源调度组成一台虚拟的“超级计算机”，来完成较为复杂的科学计算任务。而云计算强调面向用户的按需服务，服务之间可以形成组合。云计算最基本、最重要的应用场景是，通过一个相对集中的信息资源池来服务大量分散的用户，通过 web 服务和应用来满足大众用户的个性化、多样化的服务需求。可以看出，两者最明显的共同之处在于“虚拟计算”和“资源共享”，即两者都强调用虚拟化方式对计算机资源进行共享，然后提供给用户使用，以提高计算机资源的利用率。关于两者的差异，归纳如下：

（1）云计算以集群计算为主，不同计算节点和计算集群可以服务不同服务对象；而网格计算主要以计算任务的并行化分配为主，通过作业调度模块将作业分配到各个物理资源中并行处理。

（2）云计算面向多样的大众服务需求，承认节点在原理、规模、能力上的差异性，节点之间的资源共享是依靠服务之间的互操作来实现；网格计算通常利用中间件屏蔽系统的异构性，使资源以同一方式共享给用户。

（3）云计算是一种商业计算模式，向用户提供尽最大努力的多租赁服务，用

户按需使用付费；而网格计算依赖组织之间的协作式运营，没有明显的商业模式。

网格计算的初衷是为了解决高性能计算能力不足这一问题，在需要大量计算能力和数据处理能力的应用场景下，具有重要价值。而云计算则面向用户，服务大众，追求高效实用，需要商业利益的保证来推动其运营。

2.2 任务调度概述

云计算需要面临大量复杂的计算任务，如服务计算、变粒度计算、不确定计算、人参与的计算甚至于物参与的计算。完成这些任务需要解决一系列的挑战性问题，诸如如何应对计算过程中的适应性和不确定性问题、如何在大众参与下进行人网交互计算。

2.2.1 任务调度关键问题

云数据中心资源调度关键问题包括以下几个方面：

（1）调度策略：由云数据中心基础设施拥有者跟管理者制定，属于调度管理资源的最底层策略。

（2）优化目标：云调度管理系统需要根据本数据中心的特点确定优化目标。目前有以调度性能、用户服务质量、运营成本控制等为优化目标。

（3）调度算法：好的调度算法可以按照优化目标在较短的时间内产生调度结果，并且算法自身消耗资源较少。一般来讲，调度问题是 NP 难问题，需要极大的计算量而且不能通用，一般普遍采用近似优化的调度算法。

（4）调度算法的系统架构：与云数据中心采用的基础技术架构紧密相关，目前多是采用多级分布式调度体系结构。

2.2.2 任务调度流程

云计算实际上也是一种基于集群的计算模式，云计算采用虚拟化技术，让用户按需索取资源，对用户来说这些资源是透明的。这就决定了云数据中心的任务调度是二级调度模式，第一级是用户任务到虚拟机资源的调度，第二级是虚拟机资源到物理机资源的调度。如图 2.1 所示：

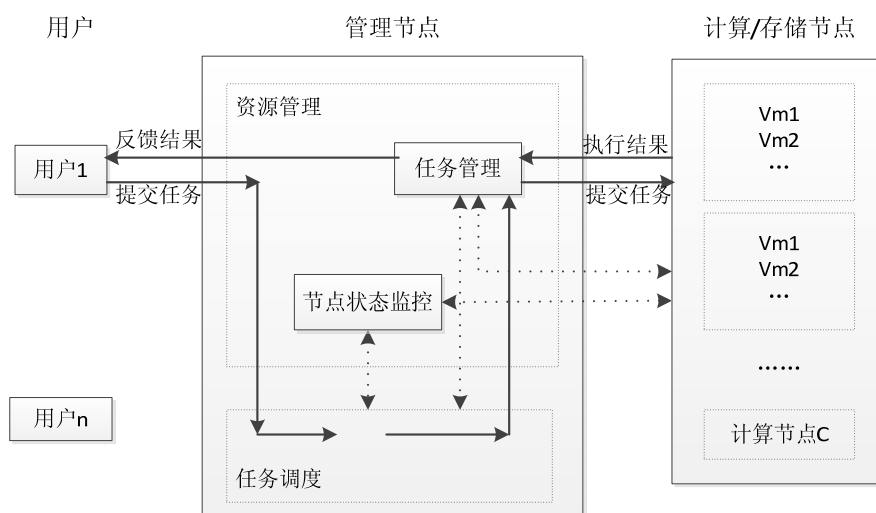


图 2.1 云计算的调度流程

Fig. 2.1 Cloud computing scheduling processes

云计算这种商业计算模式，用户按需使用付费，一级调度在尽量满足用户需求的情况下，努力提高虚拟机资源利用率；二级调度保证物理机负载均衡的同时，尽量提高物理机利用率。两级调度相互配合，目标是提高用户满意度和增加云服务提供商收益。

2.3 现有任务调度算法

从任务调度的实时性，调度策略可以分为批调度（batch scheduling）和在线调度（online scheduling）。在批调度中，任务被提交后不是立即执行，而是放入任务集合中直到调度周期或者某一事件发生后统一调度。而在线调度是任务提交后立即调度执行。批调度由于获得了较多的信息，因此调度时具有针对性，而在线调度具有较强的实时性。本论文主要研究云计算中的批调度算法。

从任务模型的角度来看，一般可以分为两类：独立任务调度（independent task scheduling）和相关联任务调度（associated task scheduling）。前者重点研究在异构的集群环境下如何调度独立任务集合，使得系统整体性能或者其他调度目标得到提高。后者着重研究如何在一个处理机集群中调度和分配多个相互关联任务，一般所追求的目标是最小完成时间。

在本文的第三章中重点研究的是独立任务批调度策略；在第四章中重点研究的是相关联任务的调度问题。

2.3.1 独立任务调度算法

（1）轮转调度算法（RR）

轮转调度算法属于在线调度，就是以轮转的方式依次将新的连接请求调度到

不同的服务器，该算法的最大优点在于简单易行，但不适用于每个节点性能不一致的情况。因此，为克服轮转调度算法的不足，又提出了加权轮转调度算法(wRR)，用相应的权值表示计算节点的处理性能，权值大的计算节点会被赋予更多的链接请求，最终，各计算节点的链接数趋向于节点对应的权值比例。

(2) 最小链接算法 (LC)

最小链接算法属于在线调度，用一个负载均衡器登记各个计算节点的链接数；当用户请求到达时，负载均衡器把该连接请求分配到当前连接数最少的物理节点上，同样的，LC 算法没有考虑物理节点处理能力不一致的情况，为克服这种不足，又提出了加权最小链接算法 (WLC)，用相应权值代表计算节点的处理能力，将连接请求分配到连接数与权值比最小的计算节点。

(3) Min-min 算法

Min-min 算法属于批调度算法。首先计算出任务集合中每个任务在各个处理机上的预计执行时间，找出每个任务的最小执行时间以及与其对应的处理机节点形成的映射对，然后从中找到具有最小执行时间的任务-处理机映射对，将该任务分配到对应的处理机上执行，分配完成后，从任务集中删除该任务，更新处理机就绪时间。重复上述过程，直到任务集中的任务都调度完成。算法的调度目标是任务集合的完成时间最短。该算法优点是简单快捷，每次通过寻找两次最小值，将任务分配到当前完成该任务时间最早的处理机上。该算法的缺点是大任务会需要较长的等待时间，且更可能被分配到到处理能力较差的处理机上，小任务会较快调度且获得较好的处理机，这将导致系统负载的不均衡。

(4) Max-min 算法

Max-min 算法属于批调度算法，是在 Min-min 调度算法基础上提出来的。区别是该算法找到每个任务的最小完成时间的任务-处理机映射对之后，总是选择其中最大的任务-处理机映射对进行映射。在异构的集群环境中，当小任务数量比大任务数量多很多时，该算法的调度结果优于 Min-min 算法，因为它倾向于优先调度大任务，使得大任务可以和其它小任务并行执行，但可能造成小任务推迟调度。

(5) Sufferage 算法

Sufferage 算法属于批调度算法。其调度思想是：在为任务分配计算资源时，若发生资源竞争，计算各个任务的执行损失，将资源分配给执行损失最大的任务。任务的执行损失定义为任务的次小完成时间与最小完成时间之间的差值，发生竞争时，必须把资源分配给估计执行损失最大的任务，否则将遭受最大损失。

(6) 遗传算法 (Genetic Algorithm, GA)

遗传算法^[9]是一种智能化算法，应用于调度问题时，属于批调度算法。遗传算法 GA 是模拟优胜劣汰的生物进化过程实现的一种搜索算法，通过交叉、变异操

作, 来实现寻优过程。GA 算法是从一个初始种群开始迭代更新, 该种群由一定数目的个体组成, 每个个体实际上就是问题的一个解, 个体由经过基因编码的染色体构成, 初始群体随机生成。在每次循环迭代中, 根据适应度函数值评价种群所有个体成员; 然后, 从当前种群中用概率方法选取适应度值最高的一部分个体到下一代种群中; 同时, 以某种交叉概率方法从种群中选择出一些个体随机搭配成对, 对每对个体以某种规则交换他们之间的部分基因, 称为交叉操作, 另外以某种变异概率从种群中选出一些个体以某种规则改变他们的基因, 称为变异操作。重复上述步骤, 直到达到迭代次数或满足终止条件。

GA 算法具有的优势: 首先, 它能同时搜索解空间中的多个解, 具有并行处理能力, 因此在大规模并行分布式处理当中能够应用; 其次, GA 的交叉变异操作使得算法陷入局部最优解的概率较小, 能较好的收敛到全局最优解; 然而, GA 算法存在搜索效率低、容易过早收敛的缺点。

(7) 粒子群算法 (Particle Swarm Optimization, PSO)

粒子群算法^[11]是一种智能调度算法, 属于批调度算法。PSO 算法是由 Dr.Eberhart 和 Dr. Kennedy 提出的一个智能优化算法, 通过模拟鸟类的觅食过程实现寻优。PSO 算法从一群初始粒子开始迭代更新, 为每个粒子初始位置和速度赋值; 然后, 计算每个粒子的适应度函数值, 比较粒子当前位置和粒子目前最优位置 pb 的优劣, 若粒子当前位置较好, 则更新粒子最优位置 pb 为当前位置, 否则不更新; 然后比较种群全局最优位置 gb 与每个粒子目前最优位置 pb 的优劣, 更新种群全局最优位置 gb 。最后, 更新各粒子的速度和位置。重复上述步骤, 当达到迭代次数或满足终止条件, 终止算法。

粒子群优化算法存在诸多优点, 例如算法参数较少, 且收敛速度较快等。但缺乏交叉变异能力, 容易导致粒子群陷入局部最优解。与粒子群算法相比, 遗传算法因存在交叉因子和变异因子, 其全局搜索能力相对较强, 但是收敛速度较慢。鉴于以上原因, 在本文第三章, 提出一个基于 PSOGA 的融合算法进行独立任务的调度。把遗传算法的交叉变异等操作与粒子群算法相融合, 以克服粒子群算法缺乏交叉变异操作引起“早熟”的缺点。

2.3.2 相关联任务调度算法

一般, 任务之间存在依赖关系的作业用有向无环图 DAG (Directed acyclic Graph) 来表示。具体呈现为当前任务节点不能在得到所有父任务节点执行结果之前开始启动执行。

(1) 表调度算法

表调度算法主要由两个独立步骤构成: (1) 确定任务优先级别: 把 DAG 任务

图中的任务按照某种规则确定其优先级，根据优先级别大小由高到低形成一个调度队列；（2）处理机选择阶段：从调度队列中依次取出任务，并将该任务分配到使其完成时间或开始执行时间最早的处理机上，直到调度队列为空。表调度通常比较实用，相比其他算法，具有较低的时间复杂性。

HEFT 算法是表调度算法的代表之一，是用来处理异构环境下处理器数目有限的任务调度算法。

①任务节点优先级的确定：从 DAG 任务图的出口任务开始，自下而上递归的计算每个任务节点的优先级，由于是在异构计算环境下，因此优先级 $SL(v_i)$ 的计算是基于任务节点的平均计算代价和通信代价的。任务队列根据优先级降序排列， $SL(v_i)$ 值最大的任务节点排在最前面，从 $SL(v_i)$ 的定义来看，这种排列方式基本满足了任务间的优先级关系。

②处理机选择阶段：处理机分配原则就是把任务节点分配到使得当前任务完成时间最早的处理机上。处理机 p_k 的最早可用时间是处理机 p_k 执行完分配给它的最后一个任务的时间。另外，HEFT 调度算法为了提高处理机利用率，采用了额外插入策略，只要满足下面两个条件，就可以将未调度任务节点插入到某个处理机的两个已调度完毕的两个任务之间的空间时间槽之间：（1）不违反任务间优先级别约束关系；（2）处理机空闲时间槽大于带插入任务在此处理机上的执行时间，即：插入后，不能影响已调度任务的执行。

HEFT 算法具体描述如下：

HEFT 调度算法：

- (1) Set the computation costs of tasks with mean values and communication costs of edges.
 - (2) Compute $SL(v_i)$ for all tasks by traversing graph upward, starting from the exit task.
 - (3) Sort the tasks in a scheduling list by nonincreasing order of $SL(v_i)$ values.
 - (4) While there are unscheduled tasks in the list do
 - (5) Select the first task v_i from the list for scheduling.
 - (6) For each processor p_k in the processor-set ($p_k \in P$) do
 - (7) Compute $ft(v_i, p_k)$ value using the insertion-based scheduling policy.
 - (8) Assign task v_i to the processor p_k that minimizes $ft(v_i, p_k)$ of task v_i .
 - (9) Endwhile.
-

HEFT 调度算法时间复杂度为 $O(mn^2)$ ，其中 m 为系统中处理机节点的总个数， n 为任务节点的总个数。HEFT 调度算法的缺陷在于虽然在步骤二采用了任务插入策略，但是一个任务节点只能被分配到一个处理机节点上，没有采用复制冗余任务，只能减少当前任务与其中一个后继任务的通信开销，而不能优化与其他后继

任务的通信开销，失去了产生更短调度长度的机会。尤其当任务节点间通信量较大时，不足更加明显。

CPOP 算法也属于表调度，CPOP 算法确定任务节点优先级的方法和分配处理机的策略都与算法 HEFT 略有不同。

确定任务优先级：先计算任务的 $BL_h(v_i)$ 值（从任务节点 v_i 到出口节点的最长路径长度）和 $TL_h(v_i)$ 值（从入口节点到任务节点 v_i 的最长路径长度），两者之和作为任务节点的优先级值。任务的优先级等于 DAG 任务图的关键路径长度的任务称为关键任务，其余任务称为非关键任务。

分配处理器阶段：关键任务只能被分配到关键路径处理机上，而非关键任务可以被分配到任意处理机上。关键路径处理机是指所有关键路径任务在其上执行时，计算代价之和最小的处理机。调度任务时，若是关键任务，则分配给关键路径处理机；若是非关键任务，则依照 HEFT 调度算法选择处理机。

CPOP 调度算法时间复杂度也为 $O(mn^2)$ ，相对于 HEFT 算法有了一定改进，但也没有考虑如何减少任务节点间的通信开销。

（2）基于任务复制的调度算法

该类算法基本调度思想是冗余的复制 DAG 任务图中的任务到某些处理机上执行，以减少处理机间的通信开销。将具有数据通信的两个任务调度到同一个处理机上执行，由于他们共享此处理机所有资源，就可以避免数据传递，从而减少因等待前驱任务节点执行结果的到来的时间。采用该类思想的调度算法主要区别在于选择哪些任务节点进行复制。该类算法的缺点是时间复杂度较高，一般用于处理机数目不受限的同构计算环境。

HCPFD 算法^[27]属于基于任务复制的调度算法，在 2004 年，Tarek Hagras 和 Jan Janecek 提出了 HCPFD(Heterogeneous Critical Parents with Fast Duplication)算法。首先，该算法将调度队列中的待调度任务分配到使其完成时间最早的处理机上；然后，判断该处理机的空闲时间槽是否满足调度该候选任务的关键父任务的条件，若满足冗余复制约束条件，则将该候选任务的关键父任务冗余的调度到此处理机空闲时间槽上，提前该候选任务的开始执行时间；以此类推，直到待调度队列中任务为空。HCPFD 调度算法的缺陷在于只考虑冗余复制待调度任务的关键父任务，而没有考虑其它的父任务，没有进一步利用处理机的空闲时间槽来降低任务间的通信开销，尤其当 DAG 任务图中通信开销比较大时。

HNDP(Heterogeneous N-predecessor Decisive Path)算法^[28]是由 Sanjeev Baskiyar 和 Christopher Dickinson 两人在 2005 年提出的。该算法修改了 HCPFD 算法的冗余复制任务策略，不仅复制关键父任务，而且考虑复制关键父任务的关键父任务，直到违反任务间的优先级约束关系或处理机空闲时间槽为空。HNDP 算法相比

HCPFD 算法提高了处理机节点的利用率,进一步提前了任务节点的开始执行时间。

(3) 基于任务聚类的调度算法

该类算法的思想: 首先, 把 DAG 任务图中所有任务节点映射到处理机数目不受限的资源环境上; 然后, 选择任务进行聚类, 在每个聚类循环中把上一步的一些集群合并成新的集群; 若两个任务被分到了同一个集群中, 则表示它们在同一处理机上执行。除了执行聚类外, 还要对任务节点开始执行时间进行排序。

(4) 基于随机搜索的调度算法

该类算法思想是结合之前搜索结果的经验知识和特定随机搜索特点, 产生新的搜索结果。该类算法的主要代表是 GA 算法和模拟退火算法。GA 算法比通常的启发式算法性能要好, 但是, 它们往往时间复杂度较高, 并需要一些控制参数。此外, 适用于某个 DAG 任务图的控制参数往往不适用于另一个 DAG 任务图, 即: 对于新的 DAG 任务图, GA 算法需要较长的训练学习时间。

2.4 本章小结

本章首先介绍了云计算相关的一些概念和技术, 接着介绍了任务调度技术及其分类。然后, 针对独立任务调度、相关联任务调度, 详细介绍了具有代表性的调度算法。并分析了存在的一些问题, 在下面的章节介绍本文提出的改进方法。

3 基于粒子群遗传算法的独立任务调度算法

根据调度任务时时间的不同，一般可分为在线调度和批调度两种方式。在线调度是当用户请求到达时，立即给予响应。而在批调度中，用户任务请求到达后并不是立即得到服务，而是形成一个任务集，等到某个事件发生或者某个时间间隔后再一起调度到虚拟机资源上。本章节是对云计算中独立任务的批调度进行研究，在线调度不在本章节考虑范围中。

3.1 独立任务调度模型

云计算中的独立任务调度涉及两个方面：独立任务和处理资源。云计算环境是面向多用户的，实时的会有多用户向云系统提交大量的任务。若用户提交的任务之间没有相互依赖关系，则称作独立任务。当然，独立任务还会通过云编程模型（例如 MapReduce 模型）进行切割。云数据中心高度虚拟化，处理资源都是通过虚拟机的方式向用户提供，用户按使用虚拟机的数量和时长进行付费。下面小节对独立任务模型和云虚拟机模型进行详细了详细说明。由于在本文中任务和虚拟机属性值的单位并不是要考虑的重点，所以默认量化为了物理量，相加相乘时，都默认为可以相操作。

3.1.1 独立任务模型

设 $T = \{t_1, t_2, t_3, \dots, t_n\}$ 为独立任务集合， $n = |T|$ 为任务总数；用一个四元组来表示任务 t_i 的参数，即

$$t_i = \{t_{id}, t_{length}, t_{data}, t_{res}\}$$

其中， t_{id} 标识任务编号 ID； t_{length} 表示任务预计执行长度； t_{data} 表示任务执行时所需要的相关数据； t_{res} 表示用户任务对虚拟机属性的期望值。

数据中心的虚拟机属性有很多，在本章节中，重点考察用户对数据中心虚拟机最关心的五个属性值。因此，进一步表示 t_{res} 如下

$$t_{res} = \{t_{comp}, t_{ram}, t_{bw}, t_{fault}, t_{price}\}$$

其中五个虚拟机参数，分别表示用户任务期望的 CPU 性能、内存大小、通信带宽、机器故障率、使用价格高低。

任务预期执行完成时间由任务执行时间、数据通信时间决定。因此，任务 t_i 在虚拟机 r_j 上的估计执行时间 $R(i, j)$ 可表示为

$$R(i, j) = \frac{t_{length}}{r_{jcomp}} + \frac{t_{data}}{r_{jbw}} \quad (3.1)$$

3.1.2 云计算虚拟机模型

云数据中心的计算资源都是以虚拟机的形式向用户提供，亚马逊^[29]针对计算密集型、内存密集型等任务类型，向用户提供 11 种不同类型的虚拟机，根据使用时长收费。在本章节中，采用亚马逊云数据中心虚拟机提供方式，假设云数据中心提供 K 类虚拟机模板为用户提供服务，每类虚拟机的数量根据实际需求进行合理配置。

设 $R = \{r_1, r_1, r_2, \dots, r_m\}$ 表示云数据中心的虚拟机集合， $M = |R|$ 表示数据中心提供的虚拟机总数。对每一个虚拟机 r_j ，可用一个三元组来表示，即

$$r_j = \{r_{id}, r_{type}, r_{cap}\}$$

其中， r_{id} 表示虚拟机编号 ID； r_{type} 表示该虚拟机所属类别，即 K 类虚拟机中的某一类，取值范围 $[1, K]$ ； r_{cap} 表示虚拟机资源的性能属性。

在本章节中，重点考察用户对数据中心虚拟机最关心的五个属性值，设

$$r_{cap} = \{r_{comp}, r_{ram}, r_{bw}, r_{fault}, r_{price}\},$$

其中， r_{comp} 表示虚拟机 CPU 性能， r_{ram} 表示虚拟机内存， r_{bw} 表示虚拟机带宽， r_{fault} 表示虚拟机的故障率， r_{price} 表示虚拟机使用价格。

3.1.3 云计算虚拟机综合性能

云计算的商业计算模式，决定了它是价格敏感型的。对于虚拟机执行用户任务涉及的 CPU 执行速度、内存大小、任务数据交换涉及的网络带宽，不同的性能关联不同的虚拟机价格。显而易见，提供商对虚拟机中各种资源的价值认可是不同的，因此，针对虚拟机的 5 个性能参数引入价值认可度概念。设云平台对虚拟机资源价值认可度 E 为：

$$E = \langle E_1, E_2, E_3, E_4, E_5 \rangle$$

其中， E_j ($0 < E_j < 1$) 表示云平台对虚拟机的第 j 个性能参数的价值认可度，且 $E_1 + E_2 + E_3 + E_4 + E_5 = 1$ 。

引入虚拟机性能参数价值认可度之后，就可以计算出 K 类虚拟机模板中每类虚拟机综合性能 R_G ：

$$R_G = \sqrt{E_1 (r_{comp})^2 + E_2 (r_{ram})^2 + E_3 (r_{bw})^2 + E_4 (r_{fault})^2 + E_5 (r_{price})^2} \quad (3.2)$$

根据公式 3.2 计算出每类虚拟机的综合性能 R_G 后，对云数据中心提供的 K 类虚拟机模板的 R_G 值从大到小进行排序，并根据排序结果设置虚拟机的 r_{type} 属性值，取值范围为 $[1, K]$ 。 R_G 值最高的第一类虚拟机 r_{type} 属性值设为 K ，表示此类虚拟机资源综合性能最好；排序最后的虚拟机资源 r_{type} 属性值设为 1，表示此类虚拟机资源综合性能最差。

3.1.4 任务-资源满意度距离

为用户分配什么样的资源，才能使得用户的满意度得到保证，同时云服务提供商利益也最大化。若一味的为用户分配最好的资源，显然，用户可以获得最大的满意度，但云服务提供商的收益下降了；相反，若分配较差的资源给用户，云服务提供商的收益提升了，但并不能保证用户体验。因此，本文提出任务-资源满意度距离概念来量化任务和资源的匹配问题。

任务-资源满意度距离是根据用户任务对资源的期望值与 K 类虚拟机模板之间的类欧式距离来计算，距离最小的任务-资源映射对，就最大程度保证了两方面的利益。设第 i 个任务与第 j 类虚拟机资源的满意度距离为：

$$EU_{ij} = \sqrt{\sum_{k=1}^5 E_k \cdot (U_{t_{ik}} - U_{r_{jk}})^2} \quad (3.3)$$

其中 E_k 为云计算提供商对虚拟机第 k 个属性的价值认同度； $U_{t_{ik}}$ 、 $U_{r_{jk}}$ 分别是归一化后的任务与资源的第 k 个性能参数。

3.2 任务调度需求分析和设计目标

3.2.1 需求分析

第二章中已经介绍过，在云计算中有几种不同的角色：云提供商，云服务提供商，企业用户以及个人用户。个人用户是直接使用云服务提供商提供的服务；企业用户可以直接使用云服务提供商提供的服务，例如直接使用 **Salesforce.com** 服务商提供的 **CRM**（客户关系管理系统）服务，或者租用云提供商的平台和基础设施来构建自己的私有云。

在这种商业计算模式中，“云”用户向云服务提供商缴纳一定的费用来获得对应的服务。云服务提供商记录了用户购买的服务，当用户有服务需求时，就提交服务请求，接着云系统响应用户请求，调度一定数量虚拟机为用户提供服务。云数据中心面向的用户是海量的，导致云系统所处理的业务也是海量的。因此，云环境下的任务调度就无比重要，对云服务提供商来说，一个高性能的任务调度系统可以减少任务完成时间的同时，提高云系统中的资源利用率，这样不仅可以满足用户需求，而且增加云服务提供商的收益。

3.2.2 设计目标

云计算调度的一般目标，是将独立任务集合调度到可用资源上，使得任务集合的完成时间最短且资源得到充分利用。调度问题是一个 **NP** 难问题，传统调度算法简单实用，但是并不能取得较好效果，因此，一些研究学者尝试利用启发式算法来解决这一问题，例如遗传算法、蚁群算法、粒子群算法等。文献[30]中徐骁勇

等人利用非支配排序遗传算法，将其应用于云计算的节能调度问题，重点关注节能问题；文献[31]中李健等人利用粒子群算法来解决大规模图处理任务的调度，并在文中对粒子群的相关操作进行了重新定义，文中是以调度长度和资源租赁成本为目标；文献[32]中张春艳等人提出了一个保证云服务质量的分组多态蚁群算法，最终减少平均完成时间。

目前，研究学者主要关注的是减少任务完成时间，对用户的任务需求关注较少；另外，在他们研究文献中，并没有对基础设施异构性进行关注。本文从用户需求满意度和虚拟机资源的异构性出发，通过对这两方面的改进，来优化粒子群初始粒子，提高粒子质量，加快收敛；最后融合遗传算法的交叉变异操作来扩展粒子的解空间，避免陷入局部解。

下一小节详细介绍本章节的独立任务调度算法步骤。

3.3 任务调度具体方案

在具体讨论本文调度算法之前，首先需要给出一些假设条件：（1）此调度策略属于批调度，云数据中心是以一个时间间隔内接收到的任务集合进行统一调度，比如每 2 分钟为一个调度间隔。（2）虚拟机资源实时状态已知，例如初始负载、是否就绪等，任务在每类异构资源上的执行代价已知，虚拟机资源集合是一个全互联的集群，两两节点之间都可以通信。（3）调度到同一个虚拟机上的任务，执行时遵循先进先出原则，任务一旦开始执行，就不能被抢断。

下面详细介绍本章节的算法流程。

3.3.1 对任务和虚拟机参数归一化处理

对虚拟机和任务期望的 CPU 性能、内存、带宽、故障率、价格五个属性进行归一化，根据所有虚拟机中的某项性能参数得到该项性能参数在所有虚拟机中的特权值和任务的期望特权值，例如对于计算能力，归一化到[0,1]区间后为：

$$Ur_{jcomp} = \frac{r_{jcomp} - \text{Min}(r_{comp})}{\text{Max}(r_{comp}) - \text{Min}(r_{comp})} \quad (3.4)$$

$$Ut_{icomp} = \frac{t_{icomp} - \text{Min}(t_{comp})}{\text{Max}(t_{comp}) - \text{Min}(t_{comp})} \quad (3.5)$$

其中 Ur_{jcomp} 是第 j 类虚拟机计算性能归一化结果， r_{jcomp} 表示第 j 类虚拟机的计算性能， $\text{Min}(r_{comp})$ 、 $\text{Max}(r_{comp})$ 分别为 K 类虚拟机参数 r_{comp} 的最小值和最大值。 Ut_{icomp} 表示第 i 个任务期望处理能力归一化结果， t_{icomp} 是第 i 个任务的期望处理能力， $\text{Min}(t_{comp})$ 和 $\text{Max}(t_{comp})$ 分别为任务集合的计算性能参数的最小值和最大值。

3.3.2 计算任务最佳匹配虚拟机

任务与某类型虚拟机之间的满意度距离越小，表明若用户任务选择该类虚拟机，则能获得相对较好的用户满意度；同时虚拟机资源的性能也能得到最大发挥，也就间接提高了资源的利用率，增加了云服务提供商的收益。

通过公式（3.3）计算任务-资源满意度距离，求得任务 i 与 k 类虚拟机满意度距离的最小值为 $Min(EU_{ij})$ ，则该任务的最佳匹配虚拟机为 $bVM(t_i)$ ：

$$bVM(t_i) = Min(EU_{ij}) = Min(\sqrt{\sum_{k=1}^5 E_k \cdot (U_{t_{ik}} - U_{r_{jk}})^2}) \quad (3.6)$$

3.3.3 PSO 与 GA 融合的混合 PSOGA 算法

1 粒子的编码

本文采用间接编码方式，粒子的编码长度取决于任务数目，每一个粒子都对应着一个任务调度解。假设任务数 $N=5$ ，可用虚拟机数 $M=8$ ，则粒子 $(5,7,4,7,4)$ 对应的就是一个任务-虚拟机分配方案，任务 1 分配在 5 号虚拟机，任务 2 分配在 7 号虚拟机，任务 3 分配在 4 号虚拟机上。然后，对粒子进行解码，得到 4 号虚拟机分配任务 $(3, 5)$ ；5 号虚拟机分配任务 1；7 号虚拟机分配任务 $(2, 4)$ 。

可以计算出虚拟机 j 上执行完成所分配的任务集所用时间 $F(j)$ 和所有任务总的执行完成时间 STF 分别为：

$$F(j) = L(j) + \sum_{i=1}^{task} R(i, j) \quad (3.7)$$

$$STF = \underset{j=1}{\overset{M}{Max}}(F(j)) \quad (3.8)$$

其中 $task$ 表示分配到虚拟机 j 上任务总数， $L(j)$ 表示虚拟机的初始负载， $R(i, j)$ 表示任务 i 在虚拟机 j 上的执行时间， M 表示虚拟机资源总数。

2 改进粒子初始化操作

传统的粒子群算法初始化操作是随机生成 S 个粒子，这样带来的问题往往是粒子的质量不高，影响算法的收敛速度，因此，构建一个高质量的初始种群，能在很大程度上缩短算法收敛时间。

本章节中对粒子初始化操作改进如下：

(1) 初始种群的 $S/4$ 个粒子按如下方法生成：根据公式 3.6 计算得到的任务最佳匹配虚拟机类型 j ，从虚拟机属性 $r_{type} = r_{jtype}$ 的虚拟机类中随机生成粒子，这样得到的粒子的任务满意度得到提高，且也可以提高资源利用。

(2) 剩余的 $3/4 \cdot S$ 个粒子：按传统粒子群算法随机生成，这样能使粒子保持

较大的搜索空间。

3 适应度函数

适应度函数是用来衡量粒子优劣的，我们做适当转换使适应度函数的寻优方向与适应度值增加的方向一致。定义适应度函数 $fitness$ 为：

$$fitness(i) = \frac{1}{STFi}; 1 \leq i \leq S \quad (3.9)$$

其中 $STFi$ 为第 i 个粒子的任务总完成时间。

因为本章节的算法还考虑了用户满意度和云服务提供商的收益问题，所以需要增加一个约束条件。本文考虑利用任务-资源满意度距离来作为约束条件，当任务和被分配到的虚拟机满意度距离较小时，说明用户的期望获得了满足，而且云服务提供商提供的虚拟机资源也得到了充分利用。定义一个阈值 τ 作为满意度距离的最大值，因此有

$$\tau \geq \sqrt{\sum_{k=1}^5 E_k \cdot (U_{t_{ik}} - U_{r_{jk}})^2} \quad (3.10)$$

这样任务总完成时间越短的粒子，适应度函数值越大，同时要满足满意度距离阈值 τ 的约束。问题转换为寻找一个满足约束条件的粒子，使其适应度值最大。

4 粒子速度与位置的更新

传统粒子群算法在解决连续域内优化问题取得了成功应用，但是在离散域内无法达到理想效果。因此，本章节中重新定义粒子位置、速度等相关操作。简述如下，设第 i 个粒子最好位置为 $pb(i)$ ，种群中粒子最好位置为 gb 。

$$V_k(t+1) = \alpha V_k(t) \oplus \beta(pb_k - X_k(t)) \oplus \gamma(gb - X_k(t)) \quad (3.11)$$

$$X_k(t+1) = X_k(t) \otimes V_k(t+1) \quad (3.12)$$

对公式 3.11 和公式 3.12 简单说明如下：

- (1) α, β, γ 为常数，且 $\alpha + \beta + \gamma = 1$ ； α, β, γ 的值分别被设置为 0.1, 0.2, 0.7。
- (2) $V1-V2$ 操作表示，若两个调度方案在同一维值相同，减法操作结果取 1，否则取 0，例如 $(9,1,2,5,7) - (9,2,5,5,7) = (1,0,0,1,1)$ ；
- (3) $\alpha V_i \oplus \beta V_j$ 操作表示，粒子以 α 的概率和 β 的概率更新 V_i 和 V_j 。例如 $0.1(1,0,1,1) \oplus 0.9(1,1,0,1) = (1,x,x,1)$ 其中 x 表示此维度取 0 或 1 不确定，本例中第二维 x 表示此维度以 0.1 概率取 0，以 0.9 的概率取 1。
- (4) $X \otimes V$ 操作表示，位置 X 按照 V 的策略对调度方案进行调整。例如 $(3,2,1,5,4) \otimes (1,0,1,1,1) = (3,x,1,5,4)$ ，表示对第二维进行调整。此操作仅针对位置和速度之间的相互操作。

5 融合 GA 交叉变异操作

粒子群算法的固有缺陷就是容易陷入局部最优解，所以引入遗传算法 GA 的交叉、变异操作，来防止粒子陷入局部最优解。交叉概率函数如下：

$$P_c = \begin{cases} k_1 \cdot (f_{\max} - f') / (f_{\max} - f_{\text{avg}}), & f' \geq f_{\text{avg}} \\ k_2, & f' < f_{\text{avg}} \end{cases} \quad (3.13)$$

其中 $f_{\max}$ 为种群最大适应度值， f_{avg} 为群体平均适应度值， f' 为要交叉的两个个体中较大个体的适应度值。(1)当 $f' \geq f_{\text{avg}}$ 时，应该让交叉概率 P_c 较小，防止适应度值大的个体统治群体，陷入局部最优解，设定 $k_1=0.36$ 。(2)当 $f' < f_{\text{avg}}$ 时，应该让交叉概率 P_c 较大，以重组出新的个体，扩展种群的搜索空间，设定 $k_2=0.82$ 。交叉操作采用均匀交叉，并随机产生一个位串作为交叉掩码。

变异操作：使用均匀的概率随机从种群中选取 $m\%$ 的个体进行变异，变异方式采用基本位变异，基因值在取值范围内随机变异，本文设定 $m=4$ ，即选取 4% 的粒子进行变异操作。

计算交叉、变异得到的粒子的适应度值 $fitness$ ，若 $fitness$ 值大于原始粒子适应度，则取代原始粒子加入到种群中，同时更新 pb 和 gb ；否则不进行更新。

算法流程图如下：

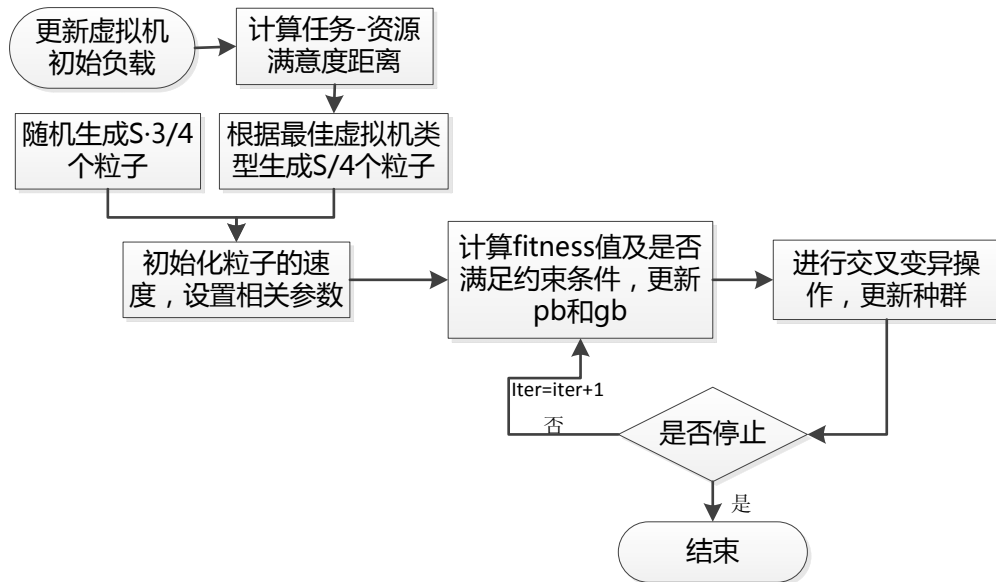


图 3.1 PSOGA 算法流程图

Fig. 3.1 PSOGA algorithm flow chart

3.4 算法性能分析

数据中心的资源数量巨大，若直接估算每个任务在每个资源上的执行时间，将耗费大量时间，本文中采用对资源进行类型编号的方法，大大减少了计算量。

设一次任务总数为 n ，计算一次调度的时间复杂度为 $O(n^2)$ 。粒子群迭代中粒子数目为 S ，迭代次数为 L ，则 PSO 算法的时间复杂度为 $S \times L \times O(n^2)$ 。PSO 算法融合了 GA 算法的交叉变异操作，两个算法是顺序结构，算法的时间复杂度并没有发生变化，但是计算量会有所增加。由于文中只对选中的少许粒子进行 GA 操作，并不会导致计算量大幅度增加，相对于扩展了解空间的优势来说是值得的。

3.5 本章小结

本章节针对云计算中的独立任务调度，综合考虑用户满意度和云服务提供商收益，提出了一种融合粒子群算法和遗传算法的调度策略。本算法的特点有：（1）通过计算虚拟机综合性能，对云环境下的虚拟机资源进行分类，同时加入任务-资源满意度距离概念，考虑了任务与资源的匹配；（2）优化了粒子群初始化粒子的质量，并通过融合 GA 的交叉变异操作扩展了寻优空间。而在实际任务调度应用中，还有许多因素需要考虑，比如负载均衡、节能等，因此还有待进一步的工作，来改进本章节提出的策略，以优化云计算任务调度。

4 基于改进表调度的相关联任务调度算法

在云计算的计算模式下，除了独立任务之外，在任务之间也可能存在相互约束的优先级关系，具体呈现为一个任务节点不能在得到它的所有父节点结果信息之前启动执行，任务之间存在依赖的作业可以用有向无环图 DAG (Directed acyclic Graph) 来表示。调度目标一般是在不破坏任务间优先级约束的前提下，将这些任务分配到云计算数据中心的各个虚拟机上，使相互约束的 DAG 任务图的总体完成时间最短。

4.1 DAG 任务建模

在云计算这样的异构集群环境下，具有相互依赖关系和数据交换的任务常用有向无环图 DAG 表示，详细定义如下：

定义 4.1：任务节点和边带权值的 DAG (node-labeled and edge-labeled) 任务图由一个四元组 $DAG = (V, E, W, C)$ 表示，其中 $V = (v_1, v_2, v_3, \dots, v_n)$ 表示任务集合， n 表示任务的总个数； $E = \{e_{ij} | v_i, v_j \in V\} \subseteq V \times V$ ，表示有数据通信的边的集合， $|E|$ 表示总共有多少通信边。集合 C 表示边上任务间通信代价的集合，记作 $C(e_{ij}) \in C$ ；如果某两个有数据通信的任务被映射到同一个虚拟机上执行， $C(e_{ij})$ 就变为 0。 W 来表示执行任务时执行代价的集合，用值 w_i 表示节点 $v_i \in V$ 的执行代价。

在 DAG 任务图中，没有前驱节点的任务称为入口节点，没有后继节点的任务称为出口节点。如果在一个 DAG 任务图中，有不止一个入口任务节点，那么我们在 DAG 图中添加一个零计算代价的任务节点，与真正的入口任务通过边权值为零的虚边相连；同样的，如果有不止一个出口任务节点，就添加一个零计算代价的任务节点，与真正的出口节点通过边权值为零的虚边相连。显然，增加零计算代价和零边权值的任务节点并不影响对 DAG 图的调度长度。所以通过处理，任何形状的 DAG 任务图，都可转化成只有一个入口任务节点和一个出口任务节点。

图 4.1 所示是一个上述定义所描述的 DAG 任务图。有时候 DAG 任务图也被称为数据流图，这是由于一个任务节点只有等到全部父任务节点的输出信息之后才能开始执行，并且在完成所有的输出信息之后才终止执行，就像一个数据流一样。由于云计算是集群环境的异构性，每个任务在各个虚拟机上的执行代价各不相同，因此由执行代价矩阵 $w(v_i, p_k)$ 给出比较合适，其中 p_k 表示虚拟机。图中圆圈外方框中的数字表示该任务节点在所有虚拟机上的平均计算代价，执行代价矩阵如表 4.1 所示：

表 4.1 执行代价矩阵

Table 4.1 Execution cost matrix

v_i	p_1	p_2	p_3
1	14	16	9
2	13	19	18
3	11	13	19
4	13	8	7
5	12	13	10
6	13	16	9
7	7	15	11
8	5	11	14

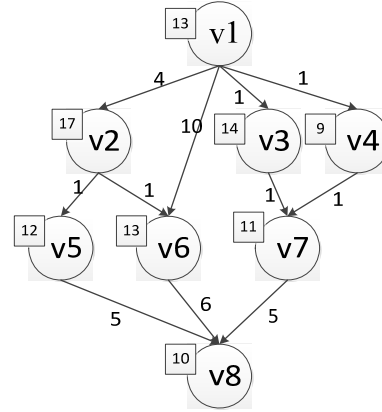


图 4.1 DAG 任务图示例

Fig. 4.1 an example of DAG task graph

定义 4.2 通信量与计算量之比 CCR : DAG 任务图中全部任务节点通信代价总和与全部任务节点计算代价之比, 称作通信计算比, 记作 CCR , 计算公式如下:

$$CCR = \sum_{e_{ij} \in E} C(e_{ij}) / \sum_{v_i \in V} \bar{w}_i \quad (4.1)$$

定义 4.3 任务节点 v_i 的父节点集合和子节点集合: DAG 任务图中任务节点 v_i 的所有前驱任务节点组成的集合称为 v_i 的父节点集合, 记作 $pred(v_i)$; 任务节点 v_i 所有的后继任务节点组成的集合称为 v_i 的子节点集合, 记作 $succ(v_i)$ 。

一般来说, 任务节点 v_i 在虚拟机 p_k 上的启动时间受到两方面因素的制约:

(1) 虚拟机的起始可用时间

虚拟机 p_k 起始可用时间指虚拟机准备就绪时间或最近一次被分配到该虚拟机上的任务的完成时间, 用 $ava(p_k)$ 表示。显然, 若虚拟机 p_k 刚准备就绪时, 有 $ava(p_k) = 0$; 若刚完成最近被分配任务的执行时, 有 $ava(p_k) = ft(v_i, p_k)$, 其中 v_i 表示最近一次被分配到虚拟机 p_k 上的任务节点, $ft(v_i, p_k)$ 表示任务 v_i 在 p_k 上的完成时间。

(2) 任务 v_i 的父任务节点执行结果传递到虚拟机 p_k 的时间

显然, 处理机 p_k 需要接收到任务 v_i 全部父任务执行结果且虚拟机可用时才开始执行 v_i , 因此, 任务 v_i 在虚拟机 p_k 上开始执行时间 $st(v_i, p_k)$ 可表示为:

$$st(v_i, p_k) = \max(ava(p_k), \max_{v_j \in pred(v_i)} (ft(v_j, p_x) + C(e_{ij})))$$

其中 $pred(v_i)$ 是指的任务 v_i 的父任务节点集合。

任务节点 v_i 在虚拟机 p_k 上的完成时间等于 v_i 在虚拟机 p_k 上的开始执行时间加上执行时间, 用 $ft(v_i, p_k)$ 表示如下:

$$ft(v_i, p_k) = st(v_i, p_k) + w(v_i, p_k)$$

定义 4.4 DAG 任务图调度长度 (makespan): DAG 任务图的调度长度指根据某一种调度算法, DAG 任务图中出口任务节点的完成的时间, 记为 SLL :

$$SLL = \min_{j=1}^m ft(v_n, p_j)$$

定义 4.5 虚拟机 p_k 空闲时间槽: 虚拟机 p_k 空闲时间槽指的是虚拟机准备就绪, 任务 v_i 在虚拟机 p_k 上等待父任务节点执行结果到来的这段时间, 记为 $slot(v_i, p_k)$ 。

有关 DAG 任务图的一些术语在表 4.2 中说明如下:

表 4.2 有关 DAG 任务图的术语说明
Table 4.2 Terms related to DAG task graph

符号	说明
v_i	DAG 图中的一个任务节点
e_{ij}	DAG 图中从任务节点 v_i 到任务节点 v_j 的通信边
$\overline{w_i}$	任务节点 v_i 在全部虚拟机上的平均执行代价
$w(v_i, p_k)$	任务 v_i 在虚拟机 p_k 上的执行代价
n	DAG 图中任务总数
E	DAG 图中边的集合
CCR	DAG 图中通信量与计算量的比值
$pred(v_i)$	任务节点 v_i 的前驱任务集合
$succ(v_i)$	任务节点 v_i 的后继任务集合
$BL(v_i)$	从任务节点 v_i 到出口任务的最长路径
$TL(v_i)$	从入口任务到任务节点 v_i 的最长路径
p_k	一台可用虚拟机
$st(v_i, p_k)$	任务 v_i 在虚拟机 p_k 上的开始执行时间
$ft(v_i, p_k)$	任务 v_i 在虚拟机 p_k 上的完成时间

4.2 异构的云计算环境

云数据中心是一个异构性很强的系统, 由大规模的服务器集群组成, 数据中心通过完善的网络计算架构, 将服务器资源相互连接, 保证服务器资源节点可以协同工作, 以结构化的方式将服务器资源整合在一起, 实现并行计算架构, 为用户提供高性能的数据处理能力和数据分析能力。计算节点异构性是指资源机各方面的差异性, 例如虚拟机 CPU 性能差异、内存以及带宽差异、访问 I/O 速度差异、

操作系统差异等等。网络的差异性是指连接计算资源节点间的网络由不同类型、不同厂家的设备组成，例如网络是以太网或者其他网络。一个异构的计算系统也可以用一个无向图模型表示，如图 4.2 所示：

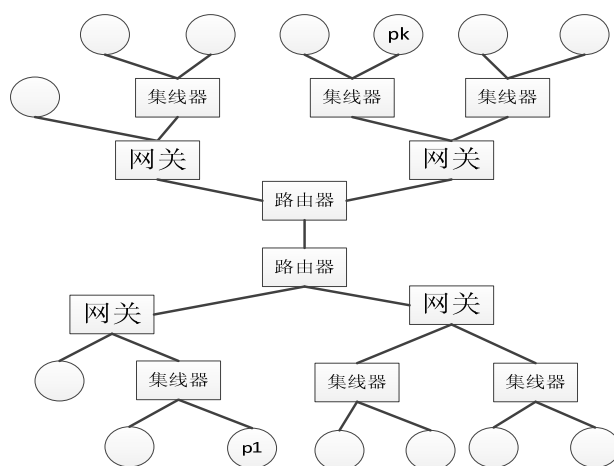


图 4.2 集群互连拓扑示意图

Fig 4.2 Cluster interconnect topology diagram

然而，如何量化计算节点处理能力的差异性。一般有两种量化方法：第一种是以计算节点的主频作为度量节点计算能力的标准，对任意类型的任务，处理机的物理主频越快，其处理任务越快；另一种是结合计算节点与任务类型来考虑，即任务的处理时间与计算节点的主频并不呈固定的比例，一些计算节点更适合处理某种类型的任务而不适合处理另一类任务，适合与否并不以其主频快慢为准。显然，第二种量化方法与实际情况更符合，本章节中我们采用第二种方法。任务在计算节点上的执行代价记作： $w(v_i, p_k)$ ，表示任务 v_i 在处理机 p_k 上的执行代价，表 4.1 列出了图 4.1 中任务节点在三个计算节点上的执行代价矩阵 $w(v_i, p_k)$ 。

4.3 基于表调度的改进的任务调度算法

4.3.1 算法假设条件

在具体讨论本章节调度算法之前，首先需要做一些假设条件。本章节讨论在云计算的异构集群环境下对相关联的 DAG 任务图的调度策略，目标是将 DAG 任务图中相关联任务调度到合适的处理器上，使得 DAG 任务图的完成时间最短。本文首先假设：（1）任务之间的依赖关系已经确定，由 DAG 任务图来表示。（2）虚拟机资源实时状态已知，例如初始负载、是否就绪等，任务在每个异构资源上的执行代价已知，虚拟机资源集合是一个全互联的集群，任意两个节点之间都可以相互通信。（3）分配到虚拟机上的任务采用非抢占方式执行。（4）每个任务只有

接收到它所有父任务执行结果后，才能开始执行；每个任务执行完成后，才能并行向它的后继任务发送执行结果。

4.3.2 问题分析

在对相关联任务的调度问题研究中，大多数静态启发式调度算法都借鉴了表调度算法的思想，HEFT 算法是静态表调度算法的代表，在本文第二章中已经详细阐述了表调度算法以及 HEFT 算法。

分析表调度策略的基本思想，第一步的关键因素是确定以什么样的规则来确定任务的优先级。一般，计算任务节点优先级别时采用的两个属性是 $BL(v_i)$ （从任务节点 v_i 到出口节点的最长路径长度）和 $TL(v_i)$ （从入口节点到任务节点 v_i 的最长路径长度）， $TL(v_i)$ 的值和任务节点的最早启动时间具有密切关系，而 $BL(v_i)$ 的值与 DAG 任务图关键路径关系密切。HEFT 调度算法实际采用 $BL(v_i)$ 来确定任务节点的优先级别，只是因为计算节点的异构性，在计算任务节点执行代价时使用在计算节点上的平均计算代价。

在确定任务优先级别时单纯的采用 $BL(v_i)$ 或 $TL(v_i)$ ，并不能取得很好的效果。因为这只是影响调度性能的一个方面，或者侧重关键路径上任务尽早开始调度，或者侧重单个任务尽早启动执行。从整体上看，这些考虑只能得到局部较优解而不是全局最优解。影响相关联任务调度的因素有很多，例如任务节点的出度和入度、任务节点之间的通信代价等等。鉴于此，我们考虑更多的影响任务调度的因素，来试图构造出更好的调度队列。在下一小节我们给出详细的计算任务优先级别的方法。

下面分析表调度的第二个步骤：依次从上一步骤的调度队列中取出就绪任务，将它们分配到使其完成时间最早的处理机节点上。大部分的调度策略模型都没有考虑任务节点之间的通信代价开销，虽然把任务调度到了使它完成时间最早的处理机上，但是任务节点间的通信开销还是会造成任务间的互相等待，最终处理机资源有过多等待时间，资源利用率和任务执行的并行性较低，导致整个 DAG 任务图的执行时间延迟。

但如果把两个相互通信的任务节点分配到同一处理机资源上时，由于他们共享了整个虚拟机资源，因此他们之间的通信代价就可以忽略不计。任务复制技术就是将某任务冗余的复制到多个处理机上，来减少任务间的通信开销，从而减少处理机空闲等待时间，提前任务节点的完成时间。任务复制手段不仅能减少任务间的通信开销，而且大大提高了用户 DAG 任务图的并行性。因此，相比不采用任务复制的调度策略，具有较好的性能，成为当前研究的热点。

现在已有一些调度策略采用表调度和任务复制相结合，利用处理机空闲时间槽降低任务节点之间的通信开销的调度策略，例如第二章介绍过的 HCFP 和 HNDP

调度算法。在 HCPFD 调度算法中，在处理机选择步骤，利用处理机空闲时间槽复制父任务，此算法只考虑复制关键父任务，因此，错失了产生更短调度长度的机会。在 HNPD 调度算法中，首先分配任务节点到完成时间最短的处理机；然后利用处理机的空闲时间槽复制父任务，若还有空闲时间，则继续复制父任务的父任务，以此类推。这两个调度算法的共同缺陷，在于考虑复制任务的时机太晚。本章节提出的调度算法在复制任务时，在以下两个方面做出改进：（1）复制任务的父任务时，考虑复制所有能够提前该任务节点最早开始执行时间的父任务。（2）提前复制任务的时机，先考虑复制任务，再在可用处理机资源中选择使任务节点完成时间最早的节点。具体的改进方法在下一小节中详述。

4.3.3 改进的表调度算法

一个 DAG 任务图由多个任务组成，每个任务的功能、角色、重要程度各不相同。因此，必须深入研究可能影响 DAG 任务图任务调度排序结果的因素，才有可能获得较好的调度结果。影响任务的因素有很多，比如任务的出度、任务的入度、任务的执行代价、任务间的通信代价等等。本文中重点考察了任务的执行代价、任务的出度和任务间的通信代价对任务优先级别的影响。

首先考察任务节点的出度值，任务节点 v_i 的出度值是指该任务的直接后继任务的数目，用 $OD(v_i)$ 表示，例如图 4.1 中任务 v_1 的出度值 $OD(v_1)=4$ ，任务 v_3 的出度值 $OD(v_3)=1$ 。分析认为出度值越大的任务节点越重要，因为任务出度值越大对后继任务执行的影响越大，该任务的优先执行越能提高后续任务的并发执行程度。直觉上，出度值为 2 的任务的重要程度是出度值为 1 的任务的 2 倍，出度值为 3 的节点重要程度是出度值为 1 的任务节点的 3 倍。对于出度值为 0 的节点需要作特殊处理，认为它的重要程度与出度值为 1 的任务节点相同。

接下来考察任务间的通信代价，对大量的 DAG 任务图进行分析，发现在调度任务时，即使把任务分配到了当前最适合它的处理机上，但是由于该任务节点必须等待其全部父任务节点的执行结果的到来，大量的时间浪费在了等待执行结果的传输上，而白白浪费了计算资源。鉴于此因素，我们又分析了任务的分支值 $B(v_i)$ 对调度的影响，任务分支值 $B(v_i)$ 即任务节点 v_i 的所有出口边通信代价之和，计算式子如下：

$$B(v_i) = \sum_{v_j \in succ(v_i)} C(e_{ij})$$

分析认为，任务节点分支值 $B(v_i)$ 越大的节点对后继任务的影响越大。它的优先执行能有效减少后继任务等待时间，从而提前后继任务的开始执行时间。因此，在考虑任务优先级别计算时，又加入了对任务节点分支值 $B(v_i)$ 的考察。

通过上面的分析。我们修改了表调度算法 HEFT 的优先级别计算公式，提出

了新的计算方法如下：

$$SL'(v_i) = \alpha OD(v_i) \bullet SL(v_i) + \beta \bullet B(v_i)$$

其中 α 和 β 是用来调整影响权重，使得计算出的优先级别能较准确反映出特定 DAG 任务图的内在关系。

确定好任务节点优先级别之后，下一步的重点就是把任务按优先顺序分配到云数据中心处理机集群上。在这一步重点研究把任务调度到哪个处理机上，使得整个 DAG 任务图的整体完成时间最短。

任务的完成时间等于任务节点的开始执行时间与执行时间之和。要分别计算把任务节点调度到每个处理机上的启动时间后，再加上执行时间才能确定，把该任务调度到哪个处理机上使得该任务完成时间最早。由于任务在各个处理机上执行代价由执行代价矩阵给出，是固定值。因此，要想任务完成时间提前，重点是使任务节点的开始执行时间尽可能的提前。

任务节点开始执行时间受两个因素制约：（1）当前处理机 p_k 的实际可用时间，即分配到此处理机上的最后一个任务节点的完成时间。（2）任务 v_i 的父任务执行结果传递到处理机 p_k 的时间，任务 v_i 需要等到全部父任务执行结果都到达处理机 p_k 时才开始执行。

若任务节点 v_i 的全部父任务执行结果已经传递到处理机 p_k ，而处理机 p_k 还在执行其他任务，此时，处理机 p_k 上必然没有空闲时间，也就不可能使用复制冗余父任务的手段来提早任务 v_i 的执行。若处理机 p_k 已经准备就绪，任务 v_i 在处理机 p_k 上等待其父任务执行结果的传递，则处理机 p_k 上必有一段空闲槽，也就是任务节点 v_i 等待父任务执行结果传递的时间，可表示为：

$$slot(v_i, p_k) = st(v_i, p_k) - ava(p_k)$$

若尽可能多的将任务节点 v_i 的父任务调度到此空闲时间槽上执行，就可以大大降低通信代价，提早任务 v_i 的开始执行时间，进而使后继任务节点提前得到调度，最终缩短 DAG 任务图的完成时间。但将任务 v_i 的父任务节点调度到空闲时间槽上执行，必须要满足两个基本条件：（1）不能违反任务节点间的优先级别约束关系。

（2）该任务节点在 p_k 上的执行时间必须小于空闲时间槽的长度，否则会由于复制冗余任务影响处理机上原先分配的任务的启动时间。因此，制定规则如下：

规则 1： 如果到目前为止，任务节点 v_i 的父任务 v_j 从未被调度到处理机 p_k 上且父任务 v_j 在处理机 p_k 上满足下式，则可以将其父任务 v_j 调度到空闲时间槽 $Slot(v_i, p_k)$ 上执行：

$$slot(v_i, p_k) \geq w(v_j, p_k) \text{ 且 } ft(v_j, p_k) < st(v_i, p_k)$$

然而并不是把任务 v_i 的任何满足规则 1 的父任务节点调度到空闲时间槽上执行，都可以提前当前任务的开始执行时间。因为决定任务节点 v_i 启动时间的是 v_i 的决策父任务执行结果到达的时间。在此，给出任务节点 v_i 的决策父任务的定义：

定义 4.6: 决策父任务 $cp(v_i)$ ：对于任意 $v_j \in pred(v_i)$ ，若满足下式，则称 v_j 是任务 v_i 的决策父任务，记为 $cp(v_i)$ ：

$$ft(v_j, p_k) + C(e_{ij}) = \max_{v_k \in pred(v_i)} (ft(v_k, p_k) + C(e_{ki}))$$

若任务 v_i 的父任务数量多于一个，则将 $ft(v_j, p_k) + C(e_{ij})$ 次大的父任务记为 $scp(v_i)$ 。

显然，应当首先考虑在处理机的空闲时间槽上复制任务 v_i 的决策父任务，才能提前当前任务的开始执行时间。下面通过性质 4.1 详细说明原因。

性质 4.1: 任务节点 v_i 被分配到处理机 p_k 上，若其决策父任务满足规则 1，则利用空闲时间槽 $slot(v_i, p_k)$ 来复制任务节点 v_i 的决策父任务，会提前任务 v_i 在处理机 p_k 上的开始执行时间 $st(v_i, p_k)$ 。

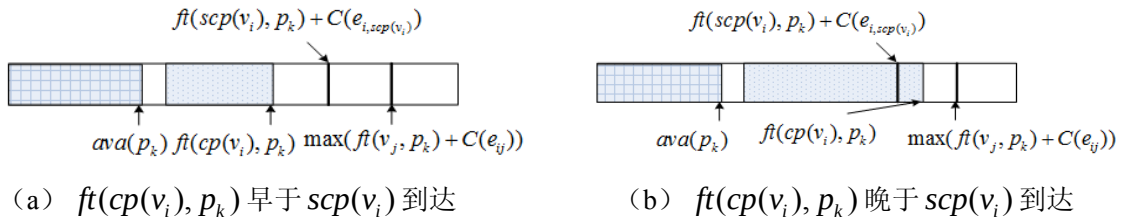


图 4.3 任务复制过程

Fig. 4.3 The process of duplication

下面简单的证明一下：

若处理机 p_k 上有空闲时间槽，则必有 $st(v_i, p_k) = \max_{v_j \in pred(v_i)} (ft(v_j, p_k) + C(e_{ji}))$ ，假设决策父任务为 v_j ，将决策父任务 v_j 插入到处理机 p_k 空闲时间槽 $slot(v_i, p_k)$ 之后。

假设任务 v_i 的父任务数目大于等于两个，则有两种情况可能出现：(1) $ft(cp(v_i), p_k)$ 早于或等于次决策父任务 $scp(v_i)$ 的执行结果到达处理机 p_k 的时间点，如图 4.3 (a) 所示，此时任务 v_i 开始执行时间 $st(v_i, p_k)$ 提前到 $ft(scp(v_i), p_k) + C(e_{i,scp(v_i)})$ 。(2) $ft(cp(v_i), p_k)$ 晚于次决策父任务 $scp(v_i)$ 的执行结果到达处理机 p_k 的时间点，如图 4.3 (b) 所示，根据规则 1 可知， $ft(cp(v_i), p_k)$ 肯定

小于 $\max(ft(v_j, p_k) + C(e_{ij}))$ ，此时任务 v_i 开始执行时间 $st(v_i, p_k)$ 提前到 $ft(cp(v_i), p_k)$ 。

假设任务 v_i 的父任务数只有一个，则将 v_i 的决策父任务插入空闲时间槽后， v_i 的开始执行时间 $st(v_i, p_k)$ 将提前至决策父任务在 p_k 上的完成时间。

假设任务 v_i 不存在父任务，则不需进行冗余复制操作。

综上所述，无论是哪种情况的出现，都将会提前任务 v_i 在处理机 p_k 上的开始执行时间 $st(v_i, p_k)$ 。

证毕。

本章节复制任务 v_i 的决策父任务步骤如下：

按任务 v_i 的父任务执行结果到达时间逆序排序，形成父任务队列。显然，父任务队列的对头就是任务 v_i 的决策父任务。判断是否满足规则 1，若满足，则将决策父任务复制到处理机 p_k 空闲时间槽上执行，然后从父任务队列删除；同时更新任务 v_i 的启动时间和处理机的空闲时间槽。此时，新的队首父任务便成为新的决策父任务，循环上述步骤，直到父任务队列为空或者新的决策父任务不再满足规则 1。性质 4.2 解释说明了终止条件。

性质 4.2： 假如任务 v_i 的第 i ($i \geq 2$) 个父任务被更新为决策父任务之后，不再满足规则 1，则冗余复制任务 v_i 其他满足规则 1 的父任务，也不能提前任务 v_i 在处理机 p_k 上的开始执行时间 $st(v_i, p_k)$ 。

简单解释一下性质 4.2：假如任务 v_i 的第 i 个父任务被更新为决策父任务，则前 $i-1$ 个父任务一定已经复制完毕。此时，任务 v_i 的开始执行时间 $st(v_i, p_k)$ 取决于第 i 个父任务执行结果到达时间，而它不满足规则 1，因此即使复制后续的父任务到上 p_k 执行，也无法提前第 i 个父任务执行结果到达时间，因此，不能提前任务 v_i 在处理机 p_k 上的开始执行时间 $st(v_i, p_k)$ 。

4.4 算法描述

本文算法首先通过改进任务优先级别的计算方法，试图构造出较好的调度队

列，为下一步的处理机分配打好基础。其次在分配处理机阶段，利用复制冗余任务的策略，来提前任务在处理机上的开始执行时间，最终达到缩短 DAG 任务图的任务完成时间的目的。

具体算法描述如下：

本文调度算法：

- 1) 计算 DAG 任务图中各任务节点 v_i 的平均执行代价 $\overline{w_i}$ 和任务间通信代价 $C(e_{ij})$ 。
 - 2) 计算 DAG 任务图中各个任务节点 v_i 的出度值 $OD(v_i)$ 和分支值 $B(v_i)$ 。
 - 3) 从下向上遍历 DAG 任务图，按公式 $SL(v_i) = \overline{w(v_i)} + \max_{v_j \in succ(v_i)} (C(e_{ij}) + SL(v_j))$ 计算任务的 $SL(v_i)$ 。
 - 4) 按公式 $SL'(v_i) = \alpha OD(v_i) \bullet SL(v_i) + \beta \bullet B(v_i)$ 计算各个任务的优先级别 $SL'(v_i)$ 。
 - 5) 按照 $SL'(v_i)$ 值从大到小排序，构成任务调度队列。
 - 6) While (未被调度的任务数 > 0)
 - 7) 从就绪任务节点队列选择第一个任务 v_i 。
 - 8) foreach ($p_k \in M$)
 - 9) 对于任意 $v_j \in pred(v_i)$ ，计算 $ft(v_j, p_x) + C(e_{ij})$ ，并根据计算结果逆序排序，形成父任务队列，队列的第一个任务节点就是当前决策父任务。
 - 10) 计算任务节点 v_i 在处理机 p_k 上的 $st(v_i, p_k)$ 和 $ft(v_i, p_k)$ 。
 - 11) 若父任务队列第一个父任务满足规则 1，则将其分配到当前空闲时间槽中，从父任务队列中删除该任务，更新 $st(v_i, p_k)$ 和 $slot(v_i, p_k)$ 。重复 (11)；若不满足，则转 (12)。
 - 12) 计算 $ft(v_i, p_k)$ 。
 - 13) end foreach
 - 14) 将任务节点 v_i 分配到使得 v_i 完成时间 $ft(v_i, p_x)$ 最早的处理机 p_x 上，更新处理机的起始可用时间 $ava(p_x)$ 。
 - 15) Endwhile.
-

HEFT 算法和 HNDP 算法的时间复杂度都是 $O(pn^2)$ ，其中 n 是 DAG 任务图总任务数目， p 为集群中处理机数目。本文算法使用表调度思想，分为构造任务节点调度队列和分配处理机两个过程。在构造调度队列时复杂度为 $O(n)$ ；在分配处理机阶段时间复杂度为 $O(pn^2)$ ，因此，整个调度算法的复杂度为 $O(pn^2)$ ，与 HEFT 及 HNDP 算法一致。

4.5 实例分析

为了验证改进算法的性能，下面对一个 DAG 任务图进行实例分析。这个 DAG 任务图包含 10 个相关联的任务，考虑把这 10 个任务调度到 3 个异构处理器上执行，如图 4.4 所示。图中圆圈内是任务编号，圆圈旁边方框中是任务在异构处理器上的平均执行代价，有向边上的数表示任务节点之间的通信代价。DAG 任务图中任务执行代价矩阵如表 4.3 所示。

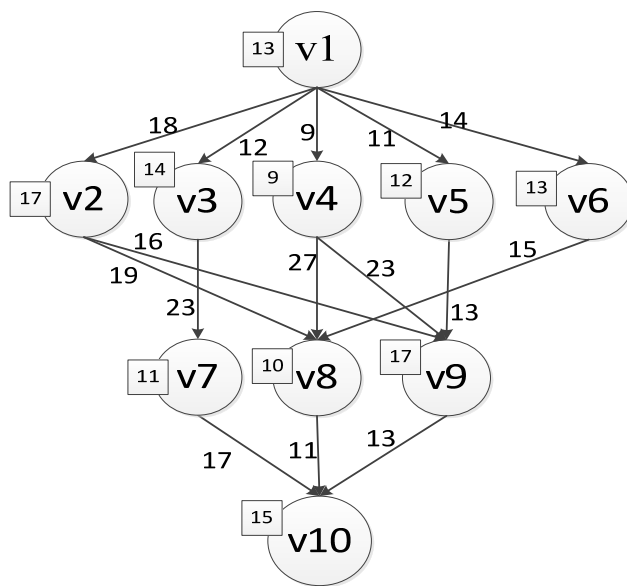


图 4.4 DAG 任务图

Fig. 4.4 DAG task graph

表 4.3 DAG 任务图执行代价矩阵

Table 4.3 an execution cost matrix of a DAG task graph

v_i	p_1	p_2	p_3
v1	14	16	9
v2	13	19	18
v3	11	13	19
v4	13	8	7
v5	12	13	10
v6	13	16	9
v7	7	15	11
v8	5	11	14
v9	18	12	20
v10	21	7	16

由图 4.4 和表 4.3，就可以根据各个调度算法的任务优先级计算方法，计算出各个任务的优先级，从而确定调度队列。在本实例分析中，将对四个调度算法进行比较，分别是：（1）经典调度算法 HEFT；（2）文献[33]改进优先级调度算法 LBP；（3）文献[34]基于任务复制的调度算法 HNBP；（4）本章节提出的改进算法。

根据图 4.4 和表 4.3，利用各个调度算法计算方法分别计算出各个任务优先级，其中 LBP 调度算法给出的是任务优先级的相对值：

表 4.4 任务优先级别计算结果

Table 4.4 The result of Task Priority

HEFT 算法		LBP 算法		HNBP 算法		本文算法	
任务	优先级	任务	优先级	任务	优先级	任务	优先级
v1	109	v1	10	v1	109	v1	609
v2	78	v2	8	v2	78	v2	191
v3	80	v3	7	v3	80	v3	103
v4	77	v4	9	v4	77	v4	204
v5	70	v5	5	v5	70	v5	83
v6	64	v6	6	v6	64	v6	79
v7	43	v7	4	v7	43	v7	60
v8	36	v8	2	v8	36	v8	47
v9	45	v9	3	v9	45	v9	58
v10	15	v10	1	v10	15	v10	15

根据表格 4.4 中计算出的优先级值，就确定了这些调度算法的调度队列。HEFT 调度算法的调度队列是 $\{n1, n3, n4, n2, n5, n6, n9, n7, n8, n10\}$ ，LBP 调度算法的调度队列是 $\{n1, n4, n2, n3, n6, n5, n7, n9, n8, n10\}$ ，HNBP 调度算法的调度队列是 $\{n1, n3, n4, n2, n5, n6, n9, n7, n8, n10\}$ ，本文调度算法的调度队列是 $\{n1, n4, n2, n3, n5, n6, n7, n9, n8, n10\}$ 。确定完调度队列之后，就可以根据各个调度算法第二个步骤的不同方式，调度任务节点到处理机上执行。

图 4.5 是 HEFT 调度算法调度示意图，调度长度为 80；图 4.6 是 LBP 调度算法的调度示意图，调度长度为 77；图 4.7 是 HNBP 调度算法的调度示意图，调度长度为 73；图 4.8 是本文算法第二步采用 HEFT 算法分配处理机时调度示意图，调度长度为 75；图 4.9 是本文算法调度示意图，调度长度为 66。其中图中阴影任务表示的是为了提前后继任务的完成时间而冗余复制的任务节点。

通过上述调度结果长度，可以看出本文算法对任务优先级的修改起到了不错

的效果，对第二阶段复制任务手段的添加也起到了很好的效果，综合比较，本章节提出的调度算法具有更佳的调度性能。

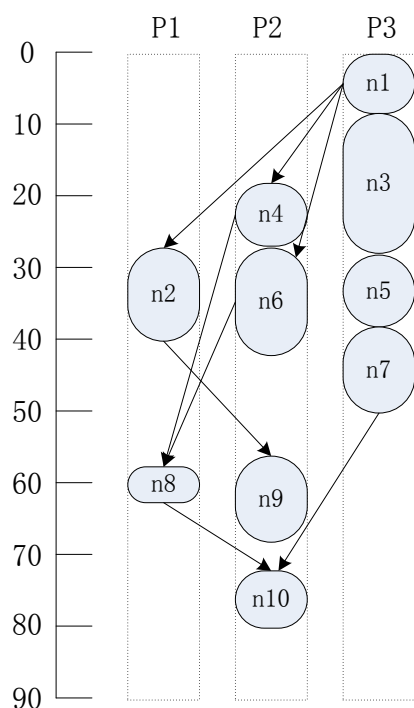


图 4.5 HEFT 算法调度结果
Fig. 4.5 Schedule of HEFT

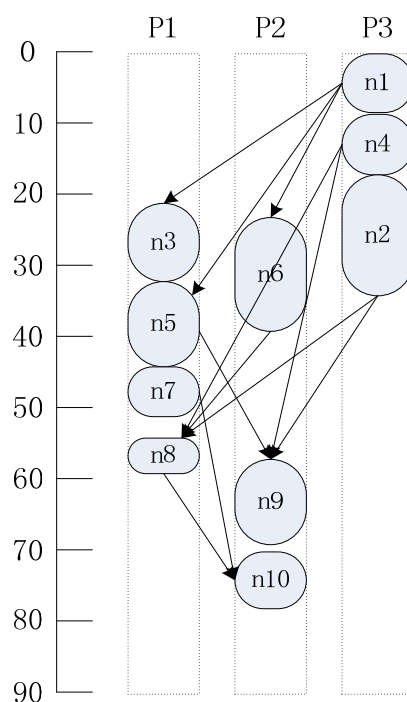


图 4.6 LBP 算法调度结果
Fig. 4.6 Schedule of LBP

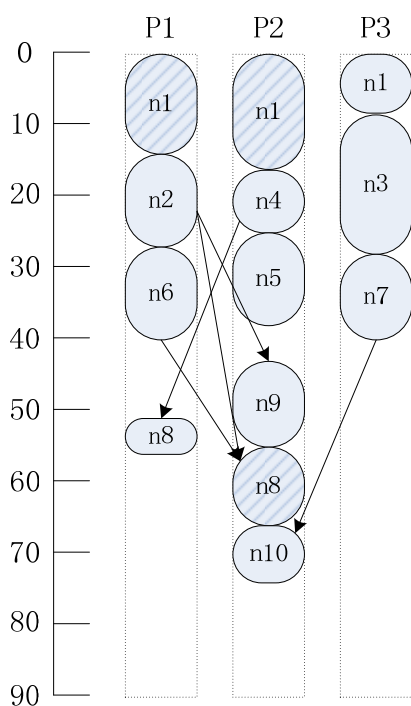


图 4.7 HNDP 算法调度结果
Fig. 4.7 Schedule of HNDP

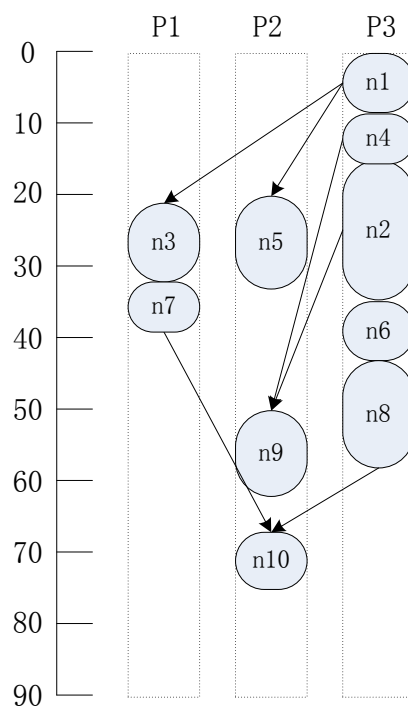


图 4.8 本文算法调度结果（无复制）
Fig. 4.8 Schedule of this article (without duplication)

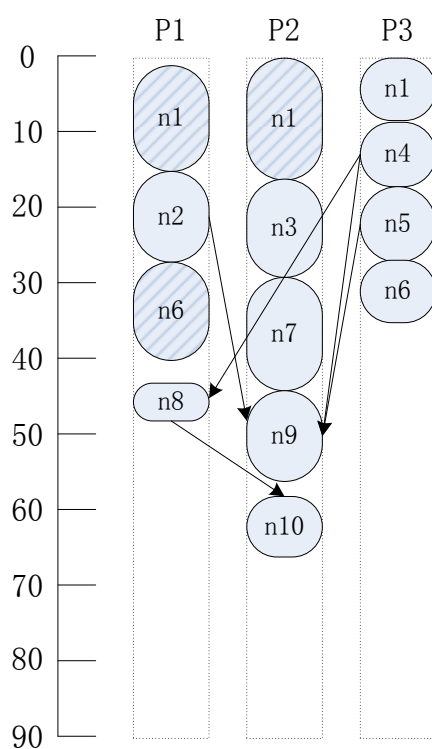


图 4.9 本文算法调度结果

Fig. 4.8 Schedule of this article

4.6 本章小结

本章首先对相关任务进行了建模，并分析了云环境的异构性；接着分析了现有调度算法存在的问题，并针对这些问题进行了改进，给出了一个改进的相关任务调度算法，最后通过一个实例和其它算法进行了比较，可以看出，本文所提出的算法具有最短的调度长度。在下一章节中，将通过具体的实验进行验证。

5 仿真实验与结果分析

为了验证本文第三章及第四章提出的算法的有效性,本章节实验采用墨尔本大学开发的云仿真平台 Cloudsim3.0^[35]来模拟云计算环境,进行验证。并结合其他算法进行对比试验,来说明本文中算法的有效性。

5.1 CloudSim 仿真工具简介

在真正部署云计算基础设施之前,进行仿真实验是非常有必要的,可以在真正部署之前发现性能瓶颈等一系列问题。2009 年,澳大利亚墨尔本大学的网格实验室宣布推出了一款称为 CloudSim 的仿真工具。CloudSim 是在离散事件模拟包 SimJava 上开发的函数库,为了克服 SimJava 的一些局限性和能模拟更复杂的场景,澳大利亚墨尔本大学的网格实验室重新设计了新的离散事件框架。

CloudSim 提供了一些新的特性^[36]: (1) 支持创建大规模云计算中心,并支持在单一物理节点上进行建模和仿真; (2) 提供一个建模平台,对数据中心、服务代理、调度策略进行建模; (3) 提供一个虚拟化引擎,以便创建和管理多个自助和协作的虚拟化服务; (4) 提供空间和时间共享两种方式为虚拟化服务分配处理机内核。

利用 CloudSim,研究人员可以专注于特定系统的设计问题,而没有必要关心以基础设施和服务为基础的与低层有关的细节。方便用户在部署自己的云应用之前进行前期评估和模拟测试,减少资金投入。用户可以反复测试他们的任务调度方案,在服务发布前将云系统的性能调节至最佳状态。

5.1.1 CloudSim 体系结构

图 5.1 详细展示了 CloudSim 的软件架构和体系结构。最底层是离散事件仿真引擎 SimJava^[37],实现了核心功能需要的高级框架;下一层是 GridSim 工具包^[38]库函数,用于支持高层软件组件的建模;在下一层 CloudSim 扩展了 GridSim 层,可以初始化和管理的系统组件组成的大型云基础设施。而且,如果云服务提供商想要研究其基础设施的不同分配策略效果时,就需要在这一层来实现他自己的策略。最上面的用户代码层,揭示与主机、应用、虚拟机、用户和服务类型等相关配置的用户代码。

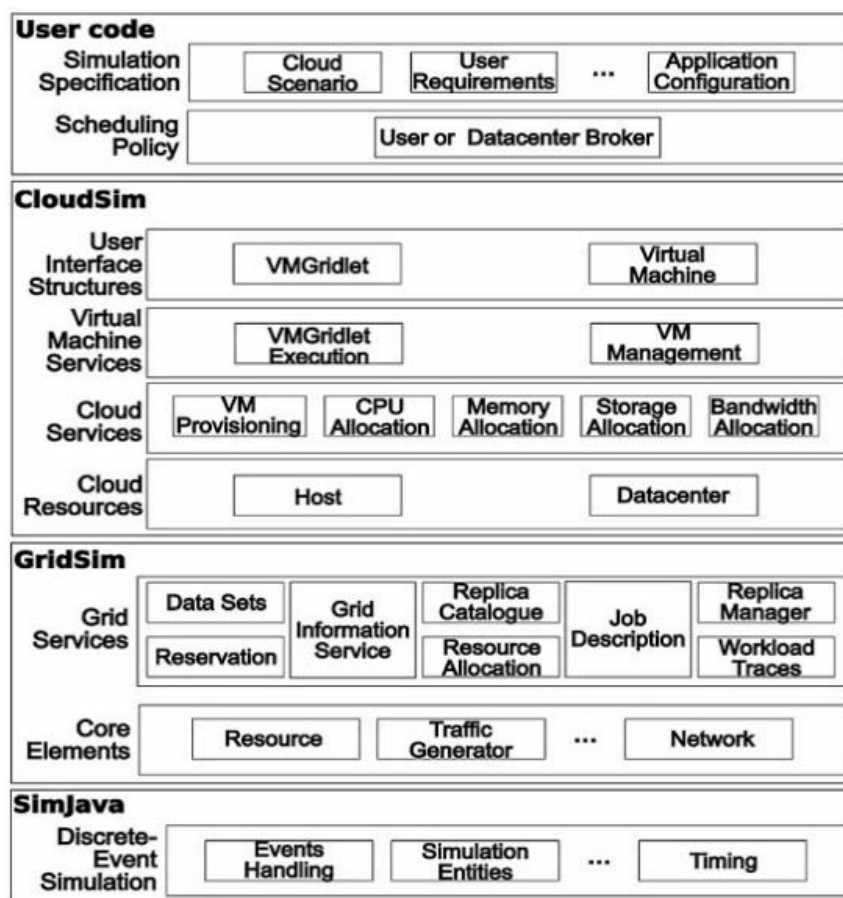


图 5.1 CloudSim 体系结构

Fig. 5.1 CloudSim Architecture

5.1.2 CloudSim 工作方式

CloudSim 工具支持对云数据中心的物理主机进行虚拟化，提供物理主机到虚拟机的映射和任务到虚拟机的映射功能。CloudSim 的 CIS（云信息服务）和 DataCenterBroker 用来实现资源发现与信息交互，是 CloudSim 的核心类。DatacenterBroker 类的 bindCloudletToVM（）方法用来指定任务到某一虚拟机上执行，用户自己开发的调度策略通过重载 bindCloudletToVM（）方法来实现。VMScheduler 类实现了虚拟机的调度分配，物理主机对虚拟机的分配由 VMProvisioner 类实现。CloudSim 的工作方式如下图所示：

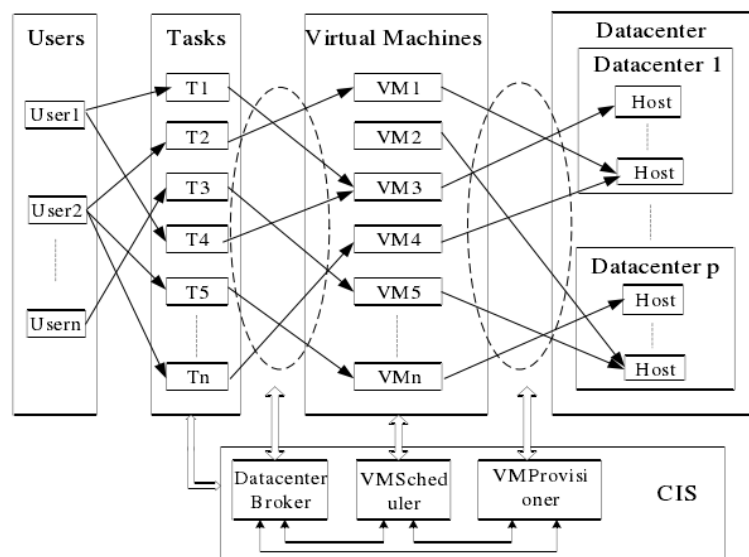


图 5.2 CloudSim 工作方式

Fig. 5.2 The working model of Cloudsim

图 5.3 描述的是 CloudSim 实体间的通信流。Datacenter 在资源创建后向 CIS 注册, CIS 为用户查询合适的云服务提供商, Datacenter 代理负责管理信息的交互过程。

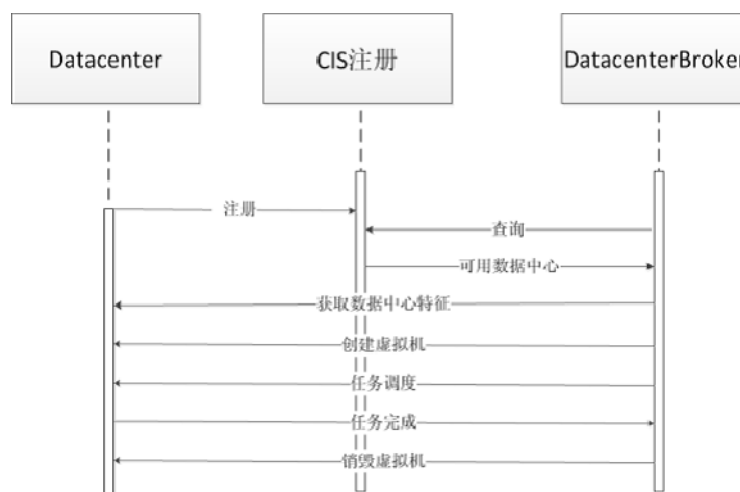


图 5.3 CloudSim 实体数据流

Fig. 5.3 The scheduling process of CloudSim

5.2 CloudSim 配置及仿真流程

5.2.1 环境配置

(1) 本机配置:

操作系统: Windows 7

内存: 4G

硬盘：300G

(2) JDK 的安装与配置

CloudSim 需要在 JDK1.6 以上版本才能安装运行，本文使用的是 jdk1.7.0_03，下载并安装。设置环境变量：新建 JAVA_HOME 变量并加入路径“C:\Program Files\Java\jdk1.7.0_03;”，在 Path 路径加入“%JAVA_HOME%\bin;”，在 CLASSPATH 路径加入“%JAVA_HOME%\lib\dt.jar; %JAVA_HOME%\lib\tools.jar;”。

(3) CloudSim 配置

本文实验使用的是 CloudSim3.0 版本。下载后解压，本文解压到了 D:\cloudsim-3.0。在 classPath 中加入路径：“D:\cloudsim-3.0\jars;”。CloudSim 安装配置成功。然后导入 Eclipse 中进行仿真实验。

5.2.2 仿真流程

CloudSim 仿真过程由以下六步组成：

(1) 初始化 CloudSim 库

```
CloudSim.init(num_user, calendar, trace_flag);
```

(2) 创建数据中心

createDatacenter() 函数用来创建数据中心，调用方法如下：

```
Datacenter datacenter0 = createDatacenter("Datacenter_0");
```

(3) 创建数据中心代理

DatacenterBroker 的功能是通过向 CIS 注册为用户选择合适的云服务提供商，并按照用户请求为其分配合适的虚拟机资源。创建方法如下：

```
DatacenterBroker broker = createBroker("Broker_0");
```

```
int brokerId = broker.getId();
```

研究人员通过扩展此类，来验证自己的调度策略，其中函数 bindCloudletToVM(int cloudletId,int vmId)的功能就是将任务调度到虚拟机上运行，用户通过重新编写这个函数，实现自己的任务调度策略。

(4) 创建虚拟机和云任务

```
//Fourth step: Create VMs and Cloudlets and send them to broker  
vmList = createVM(brokerId, 5, 0); //creating 5 vms  
cloudletList = createCloudlet(brokerId, 10, 0); // creating 10 cloudlets  
  
broker.submitVmList(vmList);  
broker.submitCloudletList(cloudletList);
```

(5) 启动仿真

```
CloudSim.startSimulation();
```

(6) 停止仿真并输出结果

```
CloudSim.stopSimulation();
```

5.3 独立任务调度算法仿真实验与结果分析

设置虚拟机类别数 $K=8$, 每类虚拟机数量服从高斯分布, 即虚拟机综合性能 R_{Gj} 较低和较高的类别数量较少, 这也符合市场机制, 虚拟机的配置通过 CloudSim 中的虚拟机资源生成器生成; 通过任务发生器的控制, 任务长度控制在 $[10000, 40000 \times \text{rand}()]$ 之间, 任务所需的相关数据 t_{data} 限制在 $[150, 200 \times \text{rand}()]$ 之间, 任务的计算需求限制在 $[500, 1000 \times \text{rand}()]$ 之间, 任务带宽需限制在 $[60, 100 \times \text{rand}()]$ 之间 (注: $\text{rand}()$ 表示取 0~1 之间的随机数), 任务对资源的期望由任务发生器随机生成, 服从正态分布。融合的 PSOGA 算法参数设置, 粒子总数 $s=50$, 最大迭代次数 $L=200$ 。PSO 算法参数设置与本文 PSOGA 算法一致, GA 算法中染色体数为 50, 最大迭代次数为 200, 交叉变异规则同本文一致。

5.3.1 评价指标

为了验证第三章提出算法的性能, 主要设置了两个评价指标: 一是任务集完成时间; 二是用户任务满意度, 单个任务满意度 J_i 的计算方法参照文献[39], 以任务期望占有资源与实际占有计算所得, 计算方法如下:

定义 5.1:

$$J_i = \ln \left(\frac{t_{i\text{comp}}}{r_{i\text{comp}}} + \frac{t_{i\text{ram}}}{r_{i\text{ram}}} + \dots + \frac{t_{i\text{price}}}{r_{i\text{price}}} \right) \quad (5.1)$$

则用户任务满意度 $U_j = \sum_{i=1}^{\text{task}} J_i / \text{task}$, 其中, task 表示任务集中任务的个数。

5.3.2 实验设计

实验一, 设定虚拟机总数 $M=200$, 每隔 2 分钟有一批任务提交给云系统, 第一批任务数为 300 个, 第二批任务数为 200 个, 第三批任务数为 200 个。重复实验 10 次, 取实验结果的平均值来降低误差。图 5.4 和图 5.5 分别显示了采用 PSO 算法、GA 算法和融合的 PSOGA 算法完成任务所用时间、满意度的对比结果。

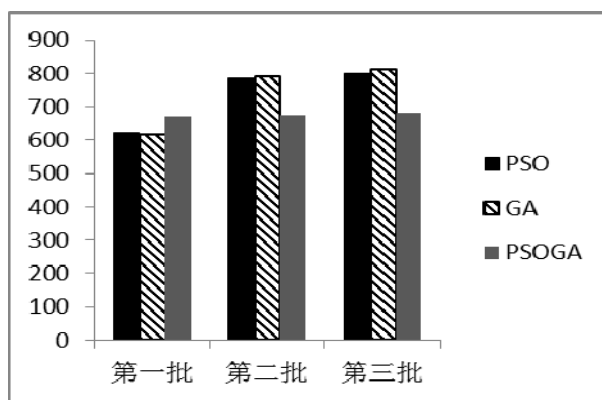


图 5.4 实验一 3 种算法任务完成时间比较

Fig. 5.4 First experiment: compare task completion time of three kinds algorithm

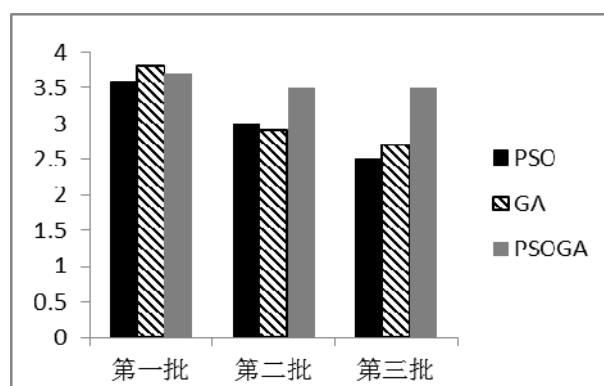


图 5.5 实验一 3 种算法用户满意度比较

Fig. 5.5 First experiment: compare customer satisfaction of three kinds algorithm

实验二，虚拟机总数 $M=200$ 不变，任务数增加，每隔 2 分钟有 400 个任务到达。

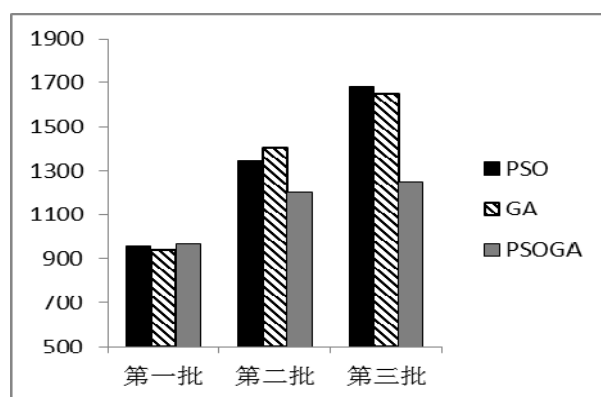


图 5.6 实验二 3 种算法任务完成时间比较

Fig. 5.6 Second experiment: compare task completion time of three kinds algorithm

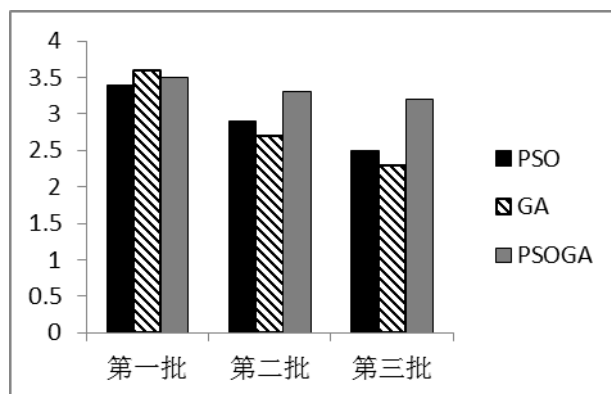


图 5.7 实验二 3 种算法用户满意度比较

Fig. 5.7 Second experiment: compare customer satisfaction of three kinds algorithm

5.3.3 实验结果分析

实验设计任务集分三批次到达，是为了模拟较真实的云计算环境。当有新任务集到达时，云环境中的虚拟机负载并不都等于零，有些虚拟机还处于任务执行状态，这种模拟环境较符合真实的云环境。

从实验一的图 5.4 结果可以看出，在第一批任务集到达时，PSOGA 算法的完成时间略高于 PSO 和 GA 算法，这是由于 PSO 和 GA 算法总是尽可能把最好的资源分配给当前任务集，当可用资源较多时，完成时间较快，而本文 PSOGA 算法还考虑任务与资源的一个匹配问题，导致完成时间略高；当第二批、第三批任务到达时，由于 PSO 和 GA 策略没有考虑任务-资源匹配，使得完成时间持续增加。图 5.5 也可以看出前期三个算法的满意度都较高，但是当系统可用资源并不是全部虚拟机时，PAO 和 GA 算法的用户任务满意度急剧下降，而 PSOGA 算法却可以基本保持在一个较高水平。实验二和实验一比较，可以看到当资源数不变，任务集的数量增加时，PSOGA 算法无论完成时间和用户满意度都更好，这说明当云环境资源数量一定时，提交到系统的任务越多，PSOGA 算法越具有优越性，这主要是 PSOGA 算法针对云任务特点和 PSO、GA 算法存在缺陷进行了一系列的改进，才达到了较好的效果。

5.4 相关联任务调度算法仿真实验及结果分析

为了验证第四章所提算法的性能，首先，利用 DAG Generator^[40]随机生成异构的 DAG 任务图，然后对生成的任务图进行处理，使之转化为具有依赖关系的任务集合，根据任务集合的属性，创建 CloudSim 云任务，重写 bindCloudletToVM() 方法，进行对比试验。

5.4.1 实验设置

1、DAG 图生成器

本文选用 DAG Generator 随机生成 DAG 任务图，为了生成任务图，需要输入一些参数，来控制生成的 DAG 任务图：

- (1) 每个 DAG 任务图所包含的任务个数 N_t 。
- (2) DAG 任务图形状控制参数 α 。DAG 任务图的高度由服从 $\sqrt{N_t} / \alpha$ 的均匀分布随机生成，结果取整。每层的任务个数由服从 $\sqrt{N_t} * \alpha$ 的均匀分布随机生成。
- (3) DAG 任务图节点的最大出度 $max_out degree$ ，出度取值范围 $[0, max_out degree]$ 。
- (4) 由于计算环境的异构性，DAG 任务图节点在不同虚拟机上计算代价不同，因此，随机生成 $N_t * Q$ 的任务-虚拟机计算代价矩阵 ETC ，其中 N_t 表示任务数， Q 表示可用虚拟机个数，矩阵元素取值范围 $[min_NodeWeight, max_NodeWeight]$ 。

(5) DAG 任务图任务节点间通信代价取值范围 $[min_EdgeWeight, max_EdgeWeight]$ 。

(6) 通过上面生成的通信代价和计算代价矩阵，可以计算出通信代价总和与计算代价总和比值 CCR 。当 CCR 较小时，说明是计算密集型任务；当 CCR 较大时，说明是通信密集型任务。

2、CloudSim 中资源参数

(1) 生成的可用虚拟机数目 Q 。

(2) 通过可用虚拟机数目 Q 与 DAG 任务图中任务总数 N_t ，可计算出虚拟机与任务数之比 m 。

5.4.2 评价指标

衡量一个任务调度算法最基本的评判标准，就是任务图的完成时间。然而，由于随机生成的不同结构的任务图有不同的属性特征，因此，应该对调度长度进行标准化变换，从而，能够对调度策略的性能进行客观准确的比较。

定义 5.3: 调度长度比率 SLR ^[41]，计算公式如下：

$$SLR = \frac{Makespans}{CPEC}$$

其中， $Makespans$ 是 DAG 任务图的完成时间，即 DAG 任务图出口任务的完成时间； $CPEC$ 是关键路径上的任务计算代价之和，而各个任务的计算代价取在每个虚拟机上执行代价的平均值，关键路径是以任务在各个虚拟机上的平均计算代价来计算。显然， $SLR \geq 1$ ，而且 SLR 越小，说明当前调度策略的 DAG 任务图完成时间越短，其调度性能越好。

定义 5.4: 性能提高百分比 $Improvement_Percentage$ ，计算公式如下

$$Improvement_Percentage = (Comparison_SLR - \text{本文算法_}SLR) / Comparison_SLR * 100\%$$

其中， $Comparison_SLR$ 是相比较算法的 SLR 值，本文算法 SLR 为本文调度算法的 SLR 值。显然，如果 $Improvement_Percentage$ 为负，说明本文算法调度性能低于相比较算法，如果 $Improvement_Percentage$ 为正，说明本文算法调度性能高于相比较算法，正值越大，说明本文算法调度性能越好。

5.4.3 实验结果分析

不同的调度算法问题假设也各不相同，在一些情况下性能表现较好，但在其他情况下性能可能较差。因此，为验证本文所提调度算法的有效性，本文选取与本文问题假设基本一致的几种算法进行比较，本文选取以下三个算法与本文所提算法进行比较，分别是：(1) 经典调度算法 HEFT，(2) 文献[33]改进优先级调度算法 LBP，(3) 文献[34]基于任务复制的调度算法 HNDP。

影响并行调度算法性能的因素有很多，本文为验证所提出的调度算法的有效性，参考了相关文献的考虑因素，决定在本实验中考察三个因素的影响，相对应的设置了三组对比实验。第一组测试 DAG 任务图中任务个数 N_t 对调度算法的影响；第二组测试虚拟机个数 Q 对调度算法的影响；第三组测试不同 CCR 对调度算法的影响。

在第一组实验中，测试 DAG 任务图中任务个数变化对算法的影响。设置虚拟机个数为 40 个，利用 DAG Generator 生成 1000 个不同的 DAG 任务图，随机设置它们的任务节点出度、计算代价矩阵、通信代价等；分别设置 $CCR = 0.5$ 和 $CCR = 10$ 来模拟计算密集型任务与通信密集型任务。图 5.8 和图 5.9 是多次重复试验的平均调度结果：

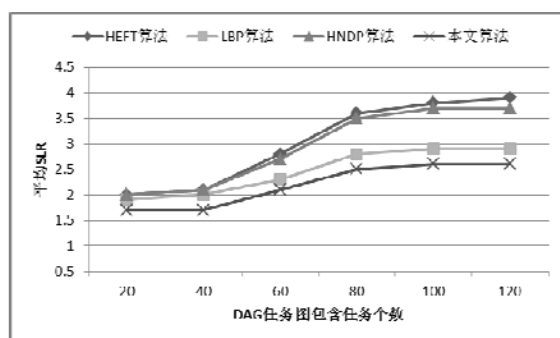


图 5.8 $CCR = 0.5$ 时，任务数变化对算法性能的影响

Fig. 5.8 $CCR = 0.5$ impacts of the task number on performance of the algorithms

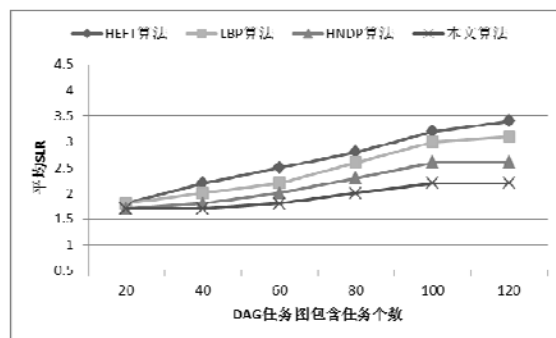


图 5.9 $CCR = 10$ 时，任务数变化对算法性能的影响

Fig. 5.9 $CCR = 10$ impacts of the task number on performance of the algorithms

从图 5.8 可以看出，随着任务个数的增加，LBP 算法和本文算法表现优于其他两个算法，这是由于当 $CCR = 0.5$ 时，DAG 任务图属于计算密集型任务，任务间的通信代价较低，冗余复制任务并不能大幅度减少任务图的通信代价，而改进任务节点优先级别的计算方法有助于更好的把任务分配到合适的虚拟机上；从图 5.9 可以看出，改变 $CCR = 10$ 时，使任务属于通信密集型任务，本文算法和 HNBP 算法优于其他两种算法，利用冗余复制任务的手段能有效降低任务图通信代价，取得了较好的效果。总体来看，本文算法性能在两种类型任务下都比较好。

在第二组实验中，测试虚拟机个数 Q 对调度算法的影响。设置任务个数为 60 个，利用 DAG Generator 生成 1000 个不同的 DAG 任务图，随机设置它们的任务节点出度、计算代价矩阵、通信代价等；同样设置 $CCR = 0.5$ 和 $CCR = 10$ 模拟计算密集型任务和通信密集型任务。图 5.10 和图 5.11 是多次试验的平均调度结果：

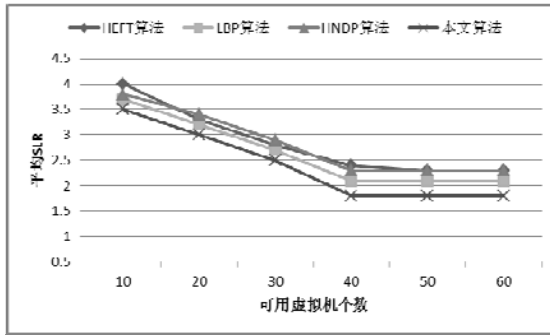


图 5.10 $CCR = 0.5$ 时，虚拟机个数对算法性能的影响

Fig. 5.10 $CCR = 0.5$ impacts of the VM number on performance of the algorithms

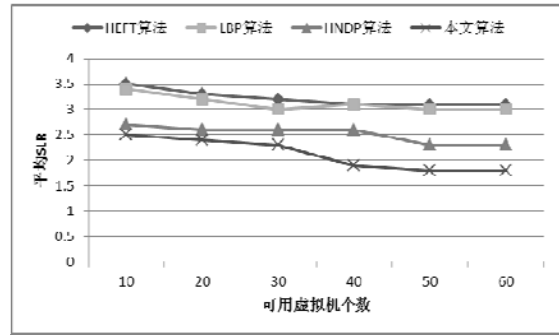


图 5.11 $CCR = 10$ 时，虚拟机个数对算法性能的影响

Fig. 5.11 $CCR = 10$ impacts of the VM number on performance of the algorithms

从图 5.10 和 5.11 中可以看出，随着虚拟机个数增减，调度算法性能都得到了提升，这是由于可用资源多了，在分配任务时有更多的选择。在调度图 5.10 计算密集型任务时，这种提高非常明显，但当虚拟机个数增加到一定程度时，性能就不会再有所增加，因为此时虚拟机资源变成了制约算法性能的次要因素。在调度图 5.11 通信密集型任务时，可以看出虚拟机的增加并没有带来明显的性能提升，因为制约性能的因素可能是传输带宽，虚拟机有大量的时间是在等待其他任务的执行结果的到来。从两个图也可以看出，本文的改进都具有一定的优势。

第三组测试不同 CCR 对调度算法的影响。利用 DAG Generator 生成 1000 个不同的 DAG 任务图，随机设置它们的任务节点出度、计算代价矩阵、通信代价等；分别设置 $m = 0.2$ 和 $m = 0.5$ ，来测试 CCR 对调度性能的影响。图 5.12 和图 5.13 是多次试验的平均调度结果：

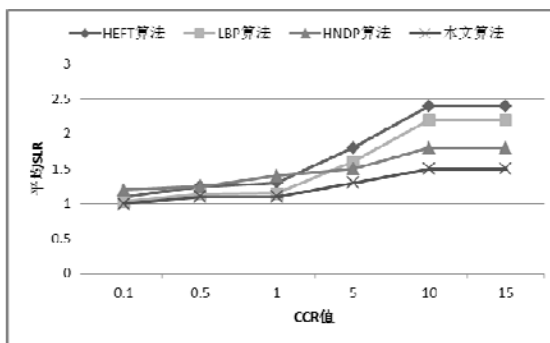


图 5.12 (a) $m = 0.2$ 时，CCR 变化对算法性能的影响

Fig. 5.12(a) $m = 0.2$ impacts of the CCR on performance of the algorithms

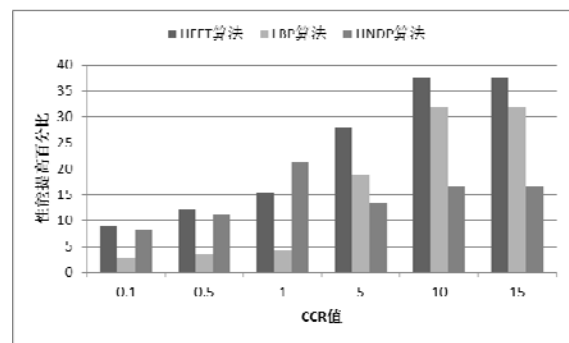


图 5.12 (b) $m = 0.2$ 时，算法性能提高比随 CCR 的变化

Fig. 5.12(b) Improvement by CCR, $m = 0.2$

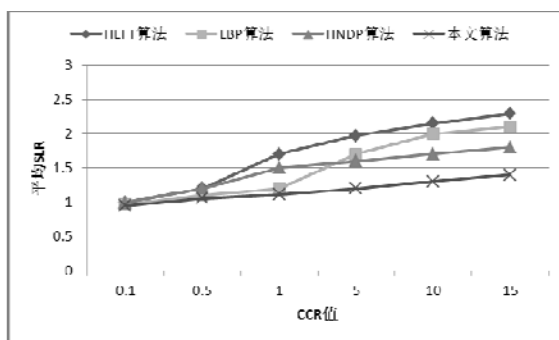


图 5.13 (a) $m = 0.5$ 时, CCR 变化对性能的影响

Fig. 5.13(a) $m = 0.5$ impacts of the CCR on performance of the algorithms

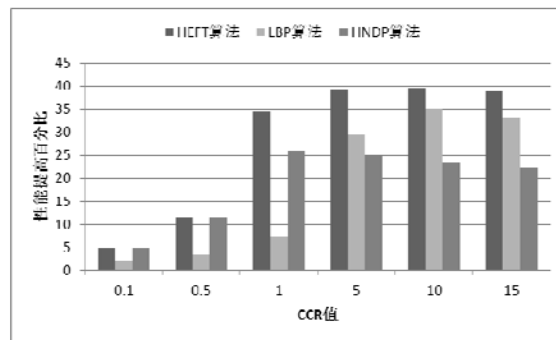


图 5.13 (b) $m = 0.5$ 时, 性能提高比随 CCR 的变化

Fig. 5.13(b) Improvement by CCR, $m = 0.5$

从图 5.12 可以看出, 当 CCR 较小时, 性能提高百分比并不明显, 这是因为 CCR 较小时, DAG 任务图属于计算密集型任务, 任务间的通信开销很少, 虚拟机上的空闲时间槽较少, 基本上不满足冗余复制父任务的条件, 但是 LBP 和本文算法对优先级的改进还是带来了一定的优势。当 CCR 较大时, 通过冗余复制父任务的手段开始带来较大的优势, 此时 HNDP 和本文算法具有较好的性能。对比图 5.12 和图 5.13 可以看出, 当虚拟机与任务数之比 m 变大时, 本文算法的性能基本上能保证稳定, 说明该算法具有较好的可扩展性。

5.5 本章小结

本章节首先介绍了 CloudSim 仿真工具, 随后介绍了 CloudSim 的配置及仿真流程。随后对第三章提出的独立任务调度算法进行了仿真实验, 分为两组实验, 两组实验结果都表明相比其他算法, 任务完成时间和用户满意度都得到了提高。最后对第四章提出的相关联任务调度算法进行了仿真实验, 分为三组实验, 分别测试了 DAG 任务图任务个数 N_t 、虚拟机个数 Q 、 CCR 三个因素对调度算法性能的影响, 从实验结果看本文算法相比其他算法具有良好性能。

6 总结与展望

云计算是传统计算和网络技术发展的融合产物，具备其他计算模式所不具有的先天优势，并且在快速发展当中，称为学术界的研究热点。用户通过租用方式来使用云提供商提供的服务或基础设施，用户提交任务，调度中心根据用户请求为其分配资源。然而复杂的云数据中心调度，已经成为阻碍云计算发展的关键问题，直接关系到云提供商收益和用户服务体验。本文分别研究了云计算中独立任务和相关联任务的调度策略。

6.1 工作总结

本文重点围绕云数据中心的独立任务和相关联任务的调度展开了研究，主要研究工作包括以下几方面：

(1) 云计算调度算法研究

详细对比分析了目前学术论文中热点研究的几种任务调度算法，并针对云计算中独立任务和相关联任务的调度策略进行了深入研究，从中发现了一些缺陷，并提出了相应的改进方法。

(2) 云计算独立任务调度算法研究

综合考虑用户满意度和云服务提供商收益，提出了一种融合粒子群算法和遗传算法的调度策略。首先，对虚拟机进行分类，引入任务-资源满意度距离、资源综合性能概念；其次，优化粒子群算法初始化粒子操作，提高初始粒子质量，并增加了任务-资源满意度距离约束；最后，融合遗传算法来扩展粒子群的搜索空间。

实验结果表明，在模拟的云计算环境下，当任务数量较少时，增加了满意度距离约束的调度策略相比其他算法优势并不明显，但持续向系统提交任务后，基本能保证用户满意度水平不变，而其他算法用户满意度持续下降；当资源数不变，任务集的数量增加时，这种效果更明显。这说明本文提出的独立任务调度策略，更适合云数据中心这种大吞吐量的环境。

(3) 云计算相关联任务调度算法研究

结合了传统表调度与基于冗余复制的调度思想。在构造任务队列时提出了新的计算方法，试图构造出更好的队列；在为任务分配处理机时，利用处理机空闲时间槽，选择性的冗余复制当前任务的父任务，减少任务间的通信代价。

仿真实验，分别测试了 DAG 任务图任务个数 N_t 、虚拟机个数 Q 、通信计算代价比 CCR 三个因素对调度算法性能的影响。从实验结果看，当虚拟机数 Q 与任务数 N_t 的比值 $m = 0.2$ ，通信计算代价比 $CCR \leq 1$ 时，性能提高百分比平均为 11% 左

右, 通信计算代价比 $CCR \geq 1$ 时, 性能提高百分比平均为 17% 左右, 说明在通信密集型类型任务中, 本文的算法优势更明显; 当虚拟机数 Q 与任务数 N_t 的比值 $m = 0.5$ 时, 即增加了系统资源数, 性能提高百分比保持稳定, 说明本文算法具有较好的可扩展性。

6.2 展望

云计算环境下的调度问题, 目前还处于初级阶段。本文分别对云环境下的独立任务和相关联任务的调度问题进行了研究, 但由于时间问题, 本文的研究工作仍然存在着一些不足之处, 在未来的工作中, 还可以从以下几方面展开:

(1) 对独立任务的调度, 本文只是考虑了任务完成时间与用户满意度两个方面, 并没有从集群的负载均衡、成本控制、能耗控制等方面进行考虑。

(2) 在对相关联任务的调度研究中, 假设了集群是全互联的环境, 没有考虑更复杂的非全互联情况, 对于调度目标只考虑了尽量减少 DAG 任务图的完成时间, 真实云环境下可能需要更多的考虑因素。

(3) 对于云环境下的任务调度, 本文中对独立任务只是考虑了批处理的调度情况, 对在线调度没有进行研究, 而对于相关联任务和独立任务也是分别研究, 并没有综合起来, 下一步研究中, 可以考虑更复杂的多种类型任务的调度问题。

致 谢

时光荏苒，硕士研究生的学习即将结束，这也代表着我在学校求学生活的结束。当初怀着怎样激动的心情来到重庆大学，想要弥补自己曾经浪费掉的青春年华，想要在这里完成自己最后的升华，不仅在学业上，还要在为人处世上。

仔细想来，六岁开始上学，至今已二十余载。这二十年间大部分时间都是在学校度过，当我敲完论文最后一个字的时候，我抬头看着屋外的骄阳，心中似有一分兴奋，似有一分担忧。兴奋的是马上有新的日子、新的生活迎接我；担忧的是自己是否能迅速融入工作环境。总而言之，心中喜忧参半。

衷心感谢我的导师王老师，对我的谆谆教诲和悉心关怀，在我硕士三年里，他给予了我生活上、学习上无微不至的关心。在我硕士论文的完成过程中，王老师提出了很多宝贵的意见和建议，并一次又一次耐心审阅。王老师严谨的治学态度，给了我很大的影响，这对我以后不管生活还是工作来说，都是一种财富。

衷心感谢我的各位同门柯军、彭伟，师兄姐梅倩、郭森、李娥、杨雄等，师弟张航、孙智国、高远等，感谢我们一起走过的苦难岁月，我会深深记住你们和这些岁月。

衷心感谢我的家人，他们把我养育这么大，还不停地关心我、支持我、鼓励我，我感谢这世上有亲情的存在，是这么的温暖。而我至今还无以回报他们，接下来的日子就是我回报他们的时候了。

我还要感谢和我一起走过六年的女友，这六年里我们风风雨雨走过，她给了我太多的鼓励和支持，我会用生命中以后的日子来爱护她。

衷心感谢所有评阅我的论文和参加答辩的各位专家教授们。

在即将结束硕士研究生生涯时，我想用一句话来总结：如人饮水，冷暖自知。

张晚磊

二〇一四年四月 于重庆

参考文献

- [1] Buyya R, Yeo C S, Venugopal S, et al. Cloud computing and emerging IT platforms: Vision, hype, and reality for delivering computing as the 5th utility[J]. Future Generation computer systems, 2009, 25(6): 599-616.
- [2] Armbrust M, Fox A, Griffith R, et al. A view of cloud computing[J]. Communications of the ACM, 2010, 53(4): 50-58.
- [3] A Vouk M. Cloud computing—issues, research and implementations[J]. CIT. Journal of Computing and Information Technology, 2008, 16(4): 235-246.
- [4] Foster I, Zhao Y, Raicu I, et al. Cloud computing and grid computing 360-degree compared[C]//Grid Computing Environments Workshop, 2008. GCE'08. Ieee, 2008: 1-10.
- [5] Ullman J D. NP -complete scheduling problems[J]. Journal of Computer and System sciences, 1975, 10(3): 384-393.
- [6] 林伟伟, 齐德昱. 云计算资源调度研究综述 [J][J]. 计算机科学, 2012, 39(10): 1-6.
- [7] 赵春燕. 云环境下作业调度算法研究与实现 [D][D]. 北京: 北京交通大学, 2009.
- [8] Etmnani K, Naghibzadeh M. A min-min max-min selective algorithm for grid task scheduling[C]//Internet, 2007. ICI 2007. 3rd IEEE/IFIP International Conference in Central Asia on. IEEE, 2007: 1-7.
- [9] Horn J, Nafpliotis N, Goldberg D E. A niched Pareto genetic algorithm for multiobjective optimization[C]//Evolutionary Computation, 1994. IEEE World Congress on Computational Intelligence., Proceedings of the First IEEE Conference on. Ieee, 1994: 82-87.
- [10] Wei G, Vasilakos A V, Xiong N. Scheduling parallel cloud computing services: An evolutionary game[C]//Information Science and Engineering (ICISE), 2009 1st International Conference on. IEEE, 2009: 376-379.
- [11] Kennedy J, Eberhart R. Particle swarm optimization[C]//Proceedings of IEEE international conference on neural networks. 1995, 4(2): 1942-1948.
- [12] 李建锋, 彭舰. 云计算环境下基于改进遗传算法的任务调度算法[J]. 计算机应用, 2011, 31(1): 184-186.
- [13] 汤小春, 刘健. 基于元区间的云计算基础设施服务的资源分配算法研究[J]. 计算机工程与应用, 2010, 46(034): 237-241.
- [14] 华夏渝, 郑骏, 胡文心. 基于云计算环境的蚁群优化计算资源分配算法[J]. 华东师范大学学报: 自然科学版, 2010, 1(1): 127.
- [15] He X S, Sun X H, Von Laszewski G. QoS guided min-min heuristic for grid task scheduling[J].

- Journal of Computer Science and Technology, 2003, 18(4): 442-451.
- [16] 孙大为, 常桂然, 李凤云, 等. 一种基于免疫克隆的偏好多维 QoS 云资源调度优化算法[J]. 电子学报, 2011, 39(8): 1824-1831.
- [17] Hermenier F, Lorca X, Menaud J M, et al. Entropy: a consolidation manager for clusters[C]//Proceedings of the 2009 ACM SIGPLAN/SIGOPS international conference on Virtual execution environments. ACM, 2009: 41-50.
- [18] Van H N, Tran F D, Menaud J M. SLA-aware virtual resource management for cloud infrastructures[C]//Computer and Information Technology, 2009. CIT'09. Ninth IEEE International Conference on. IEEE, 2009, 1: 357-362.
- [19] Nguyen Van H, Dang Tran F, Menaud J M. Autonomic virtual resource management for service hosting platforms[C]//Proceedings of the 2009 ICSE Workshop on Software Engineering Challenges of Cloud Computing. IEEE Computer Society, 2009: 1-8.
- [20] Buyya R, Yeo C S, Venugopal S. Market-oriented cloud computing: Vision, hype, and reality for delivering it services as computing utilities[C]//High Performance Computing and Communications, 2008. HPCC'08. 10th IEEE International Conference on. IEEE, 2008: 5-13.
- [21] Deelman E, Singh G, Livny M, et al. The cost of doing science on the cloud: the montage example[C]//Proceedings of the 2008 ACM/IEEE conference on Supercomputing. IEEE Press, 2008: 50.
- [22] De Assunção M D, Di Costanzo A, Buyya R. Evaluating the cost-benefit of using cloud computing to extend the capacity of clusters[C]//Proceedings of the 18th ACM international symposium on High performance distributed computing. ACM, 2009: 141-150.
- [23] Topcuoglu H, Hariri S, Wu M. Performance-effective and low-complexity task scheduling for heterogeneous computing[J]. Parallel and Distributed Systems, IEEE Transactions on, 2002, 13(3): 260-274.
- [24] Wang L, Siegel H J, Roychowdhury V P, et al. Task matching and scheduling in heterogeneous computing environments using a genetic-algorithm-based approach[J]. Journal of Parallel and Distributed Computing, 1997, 47(1): 8-22.
- [25] Ulungu E L, Teghem J. Multi - objective combinatorial optimization problems: A survey[J]. Journal of Multi - Criteria Decision Analysis, 1994, 3(2): 83-104.
- [26] Salman A, Ahmad I, Al-Madani S. Particle swarm optimization for task assignment problem[J]. Microprocessors and Microsystems, 2002, 26(8): 363-371.
- [27] Hagrais T, Janeček J. A high performance, low complexity algorithm for compile-time task scheduling in heterogeneous systems[J]. Parallel Computing, 2005, 31(7): 653-670.
- [28] Baskiyar S, Dickinson C. Scheduling directed a-cyclic task graphs on a bounded set of

- heterogeneous processors using task duplication[J]. Journal of Parallel and Distributed Computing, 2005, 65(8): 911-921.
- [29] Amazon.[EB](2009-03-01)[2012-05-17]<http://calculator.s3.amazonaws.com/calc5.html>.
- [30] 徐骁勇, 潘郁, 凌晨. 云计算环境下资源的节能调度[J]. 计算机应用, 2012, 32(7): 1913-1915.
- [31] 李健, 黄庆佳, 刘一阳, 等. 云计算环境下基于粒子群优化的大规模图处理任务调度算法[J]. 西安交通大学学报, 2012, 46(12).
- [32] 张春艳, 刘清林, 孟珂. 基于蚁群优化算法的云计算任务分配[J]. 计算机应用, 2012, 32(5): 1418-1420.
- [33] 马丹, 张薇, 李肯立. 并行任务调度算法研究[J]. 计算机应用研究, 2004, 11: 91-94.
- [34] Baskiyar S, Dickinson C. Scheduling directed a-cyclic task graphs on a bounded set of heterogeneous processors using task duplication[J]. Journal of Parallel and Distributed Computing, 2005, 65(8): 911-921.
- [35] Calheiros R N, Ranjan R, Beloglazov A, et al. CloudSim: a toolkit for modeling and simulation of cloud computing environments and evaluation of resource provisioning algorithms[J]. Software: Practice and Experience, 2011, 41(1): 23-50.
- [36] GRIDS Laboratory.<http://www.gridbus.org/cloudsim/>.
- [37] Howell F, McNab R. SimJava: A discrete event simulation library for java[J]. Simulation Series, 1998, 30: 51-56.
- [38] Buyya R, Murshed M. Gridsim: A toolkit for the modeling and simulation of distributed resource management and scheduling for grid computing[J]. Concurrency and computation: practice and experience, 2002, 14(13 - 15): 1175-1220.
- [39] 李文娟, 张启飞, 平玲娣, 等. 基于模糊聚类的云任务调度算法[J]. 通信学报, 2012, 33(3): 146-154.
- [40] Suter F. DAG generation program[J]. 2009.
- [41] Baskiyar S, Dickinson C. Scheduling directed a-cyclic task graphs on a bounded set of heterogeneous processors using task duplication[J]. Journal of Parallel and Distributed Computing, 2005, 65(8): 911-921.

附 录

A. 攻读硕士学位期间发表的论文:

- [1] 王波,张晓磊. 基于粒子群遗传算法的云计算任务调度研究.计算机工程与应用. (2013.6 录用)